

INFOSYS 722 – DATA MINING AND BIG DATA

Semester Two 2025

Iteration 3 – OSAS (Steps 1-8)

Student: Alesson Abao

UPI: aaba474

Table of Contents

1. Business Understanding (BU)	1
1.1. Situation and Objectives	1
1.2. Situation Assessment	2
1.2.1. Resources	2
1.2.2. Requirements, Assumptions, and Constraints	3
1.2.3. Risks and Contingencies	3
1.2.3.1. Inaccurate model due to limited expertise	3
1.2.3.2. Ineffective data mining techniques	4
1.2.3.3. Poor time management	4
1.2.3.4. Unforeseen events	4
1.2.4. Benefits and Costs	4
1.3. Data Mining Objectives and Success Criteria	4
1.4. Project Plan	5
2. Data Understanding (DU)	7
2.1. Collecting Initial Data	8
2.2. Describing Data	9
2.3. Data Exploration	11
2.4. Verifying Data Quality	15
3. Data Preparation (DP)	17
3.1. Data Selection	17
3.2. Data Cleaning	18
3.3. Data Construct	19
3.4. Data Integration	20
3.5. Data Reformatting	21
4. Data Transformation (DT)	22
4.1. Data Reduction	23
4.2. Data Projection	24
5. Data Mining Methods (DMM)	25
5.1. Data Mining Methods Context	25
5.2. Data Mining Method Selection	26
6. Data Mining Algorithm (DMA)	26
6.1. Data Mining Algorithm Analysis	26
6.2. Data Algorithm Selection	27
6.3. Model and Parameter Selection	29
6.3.1. XGBoost	29
6.3.2. Naive Bayes	30
7. Data Mining (DM)	31
7.1. Data Mining Tests	31
7.2. Running Model	31
7.3. Data Mining Patterns	32
7.3.1. XGBoost	32
7.3.2. Naive Bayes	34
8. Interpretation (INT)	35

8.1. Mined Patterns.....	35
8.2. Data, Results, Models, and Patterns Visualisation.....	36
8.2.1. XGBoost.....	36
8.3. Results, Models, and Patterns Interpretation.....	38
8.4. Assessment and Evaluation of Results, Models, and Patterns.....	38
8.5. Iterations.....	39
Appendices.....	40
Appendix A: Gantt Chart.....	40
Appendix B: Describing Colorectal Cancer Dataset.....	41
Appendix C: Checking for null and missing data.....	42
Appendix D: Checking for duplicates and outliers.....	43
Appendix E: Data cleaning.....	44
Appendix F: Data construct.....	45
Appendix G: Data integration.....	46
Appendix H: Data reformatting.....	47
Appendix H continuation.....	48
Appendix I: Correlation heatmap before reduction.....	49
Appendix J: XGBoost Model.....	50
Appendix J continuation.....	51
Appendix K: Naive Bayes Model.....	52
Appendix K continuation.....	53
References.....	54
Disclaimer.....	57

1. Business Understanding (BU)

A significant motivating factor for the development of the proposed study is the researcher's connection as a second-degree relative of a colorectal cancer patient who died from the disease in 2019. The study also aims to contribute to the objectives set out in the United Nations (UN) Sustainable Development Goals (SDGs). The proposal directly supports SDG 3: Ensure healthy lives and promote well-being for all at all ages. It aligns explicitly with the target to strengthen the capacity of all countries, particularly developing nations, for early warning, risk reduction, and management of national and global health risks (Long, 2022).

1.1. Situation and Objectives

Cancer remains the primary cause of mortality, with rectal and colon cancer accounting for 1.93 million cases globally (World Health Organisation, 2025). In particular, colorectal cancer (CRC) comes in second for deaths related to cancer (Matsuda et al., 2025). The development of CRC is unknown. Numerous medical research, however, have determined a number of characteristics that affect the likelihood of developing the disease (Duan et al., 2022). These characteristics can be divided into two categories: controllable and uncontrollable. Lifestyle choices, alcohol consumption, dietary habits, and tobacco use are all characteristics that an individual can control. Meanwhile, genetic predisposition, age, pre-existing non-cancer medical illness, and family history of cancer are characteristics that an individual cannot control. In medical research, there is a discourse on whether gender has a role in the development of CRC (Matsuda et al., 2025; Lewandowska et al., 2022). There is a difference of opinion, with some researchers acknowledging it while others doubting its significance. In this paper, the researcher will examine gender as a contributing factor, as does the majority of medical research.

Predictive models that calculate the probability of patient survival within a specified time frame after diagnosis can be used by healthcare systems to improve the treatment of CRC. Clinicians can use these predictive survival models to determine whether patients have a higher probability of mortality within five years and modify treatment plans to improve patient health outcomes. The influence of lifestyle, socioeconomic status, and genetic characteristics on long-term prognosis can be measured by using big data to identify predictive patterns linked to survival outcomes. In order to improve prognostic accuracy, numerous studies have created a variety of modelling methodologies.

A study on individualised risk stratification developed a comprehensive CRC risk prediction model incorporating genetic predisposition, environmental factors, family history, and personal characteristics (Zheng et al., 2020). Another study focused on improving diagnostic accuracy by integrating RNA sequencing, proteomics, and immunohistochemical image data to construct a colorectal cancer diagnostic model (Ning et al., 2024). To support early detection across diverse populations, another study utilised large-scale global dietary and demographic data to train machine learning models for CRC risk prediction (Abdul Rahman et al., 2023). A symptom-based approach was adopted in a study that analysed demographic and clinical symptom data from patients presenting with rectal bleeding to

prioritise high-risk individuals for colonoscopy in low-resource settings (Alatise et al., 2018). Lastly, a study that utilised sex, age, and sequential faecal haemoglobin concentration data from a nationwide screening programme compared logistic regression and random forest models to validate and refine CRC risk prediction tools for personalised screening strategies (Mülder et al., 2023).

The initial goals were to predict the risk of CRC development by analysing and interpreting patterns in patient datasets that were specific to each individual. However, it became clear after a more thorough examination of the selected dataset that every record relates to patients who have already received a CRC diagnosis. Since the dataset excludes those without the cancer for comparative modelling, the initial goal of predicting the risk of CRC development is less applicable.

The objectives have been improved to better suit the available variables. In particular, the dataset includes clinical, demographic, and lifestyle characteristics that can be used to calculate survival probability, as well as a variable that indicates five-year survival outcomes. This makes it possible for research to focus on predictive survival modelling, which is more in line with the overall scope of the dataset while helping in the development of tools that may assist with prognosis evaluation and treatment planning for patients with CRC.

For this proposal, the following objectives are as follows:

- Predict the probability that a patient would survive for five years after receiving a CRC diagnosis based on controllable and uncontrollable characteristics.
- Identify and analyse significant characteristics influencing survival outcomes among patients with CRC.

To be considered a success, the following business success criteria should be met:

- Identify the key variables that have a significant influence on CRC patients' five-year survival.
- Aim to improve five-year survival prediction accuracy by 1-10% with each model iteration.

1.2. Situation Assessment

This section will present considerations involving the resources needed for this proposal and its requirements, assumptions, constraints, risks, contingencies, costs, and benefits.

1.2.1. Resources

All software and hardware resources for this project will be derived mostly from the materials offered by the Data Mining and Big Data course on the Canvas learning platform, as well as those made available by the university. The dataset will be obtained through Kaggle, where numerous datasets were considered based on their relevance to the research topic. The [Colorectal Cancer Global Dataset and Predictions](#) was chosen as the primary data source since the data includes patient records with both controllable and uncontrollable characteristics, as well as information on treatment costs, tumour size, and country of residency.

Other datasets were examined, such as the [Colorectal Cancer WSI dataset](#), which contains histological images for CRC diagnosis, and the [Dietary and Lifestyle dataset](#), which consists of synthetic patient records that focus on lifestyle and nutritional characteristics relevant to early detection. These alternatives were deemed unsuitable for the purposes of this proposal.

1.2.2. Requirements, Assumptions, and Constraints

The schedule for completion of the project will align with the academic timeline established by the course coordinator. It will depend on the researcher's ability to manage time effectively and apply relevant data mining concepts and techniques. To achieve its intended purpose, the resulting predictive model must be sufficiently comprehensive to support the early detection of CRC. Consequently, the outputs must demonstrate statistical validity and be readily interpretable by healthcare practitioners to ensure practical utility in clinical decision-making. The dataset used in this study complies with the New Zealand Privacy Act 2020 requirements, as it does not contain any personally identifiable information. Additionally, the dataset is released under the MIT License, which permits unrestricted use, modification, and distribution for academic and research purposes.

It is assumed that the chosen dataset does not contain a substantial proportion of missing values and that the variables related to CRC development exhibit a statistically significant association with CRC diagnosis and risk levels. However, several assumptions remain non-verifiable within the scope of this project. These include the potential adoption of the predictive model in clinical practice, possible shifts in medical knowledge over time, and the extent to which clinicians may incorporate the model's insights into clinical decision-making processes. While the dataset initially contains 167,497 records, the number of relevant data points will likely decrease during the data processing phase due to filtering, feature selection, and clearing the chosen dataset.

1.2.3. Risks and Contingencies

In any project, several potential risks may delay or compromise the project's overall success. Implementing contingency plans becomes essential when risks or uncertainties occur, given that they provide structured responses to mitigate potential disruptions throughout the project. For instance:

1.2.3.1. Inaccurate model due to limited expertise

A key risk is that the predictive model may produce inaccurate outputs due to the researcher's limited experience in data mining and model development. Should this issue arise, the researcher will address the knowledge gap by engaging in a self-directed study, consulting relevant resources, and seeking guidance from course tutors.

1.2.3.2. Ineffective data mining techniques

Another possible risk is that the applied data mining technique may not be well-suited to the nature of the problem or that the model fails to generate meaningful insights. To mitigate this, the researcher will adopt an iterative approach, test alternative algorithms, and re-evaluate previous steps until an interpretable outcome is achieved.

1.2.3.3. Poor time management

Possible delays in the project and quality of work could be compromised due to the researcher's ineffective time management. To mitigate these risks, structured time management techniques such as the Pomodoro and scheduling tasks throughout the project will be implemented.

1.2.3.4. Unforeseen events

External events such as natural disasters, accidents, or personal emergencies may prevent the researcher from completing the project as scheduled. The researcher will notify academic staff as soon as they can reasonably do so and seek guidance on the next steps should this event occur.

1.2.4. Benefits and Costs

Ultimately, the project aims to support early detection of CRC using data mining techniques. The main benefit of this project is to help reduce mortality rates by identifying high-risk individuals before symptoms occur. It is important to note that this project's scope does not involve evaluating the various clinical screening methods available to individuals.

Regarding costs, nationwide screening programmes could significantly reduce the number of individuals developing the disease. Colonoscopy screening reduced CRC cases by 56% and 63% deaths over 40 years of research in China. While it is found to be the most effective screening for CRC prevention, it does not prove to be the most economical screening for individuals at high risk. Faecal immunochemical test (FIT) could be an alternative for individuals with low socioeconomic status and prevent CRC-related cases and deaths (Lu et al., 2024). Another report found that increasing screening rates to 80% across communities in the United States could prevent more than 200,000 deaths and reduce treatment costs by an estimated US\$12 billion by 2030 (Green & Meenan, 2020).

1.3. Data Mining Objectives and Success Criteria

The study aims to achieve the following data mining objectives:

- Develop a predictive model that accurately classifies whether a CRC patient will survive for at least 5 years with a baseline target of at least 70% accuracy.
- Preprocess and clean the dataset to eliminate missing values and outliers.
- Train 1-3 classification models and compare which works best as a predictive model.

- Evaluate models using the area under the curve (AUC), recall score, specificity, and precision.

To evaluate the effectiveness of this project, the following success criteria are as follows:

- At least one model achieves $\geq 70\%$ accuracy.
- To guarantee clinical relevance, models should also have acceptable performance in terms of recall, specificity, and AUC.

According to recent studies, machine learning models often achieve 70% accuracy in predicting the survival of CRC. For instance, Kos et al. (2024) showed that, depending on the cancer's stage and the data used, models predicting both short- and long-term CRC survival frequently reach 70% or more. This indicates that 70% is not regarded as an extremely high score but rather as a fair starting point or baseline in medical prediction tests such as this one.

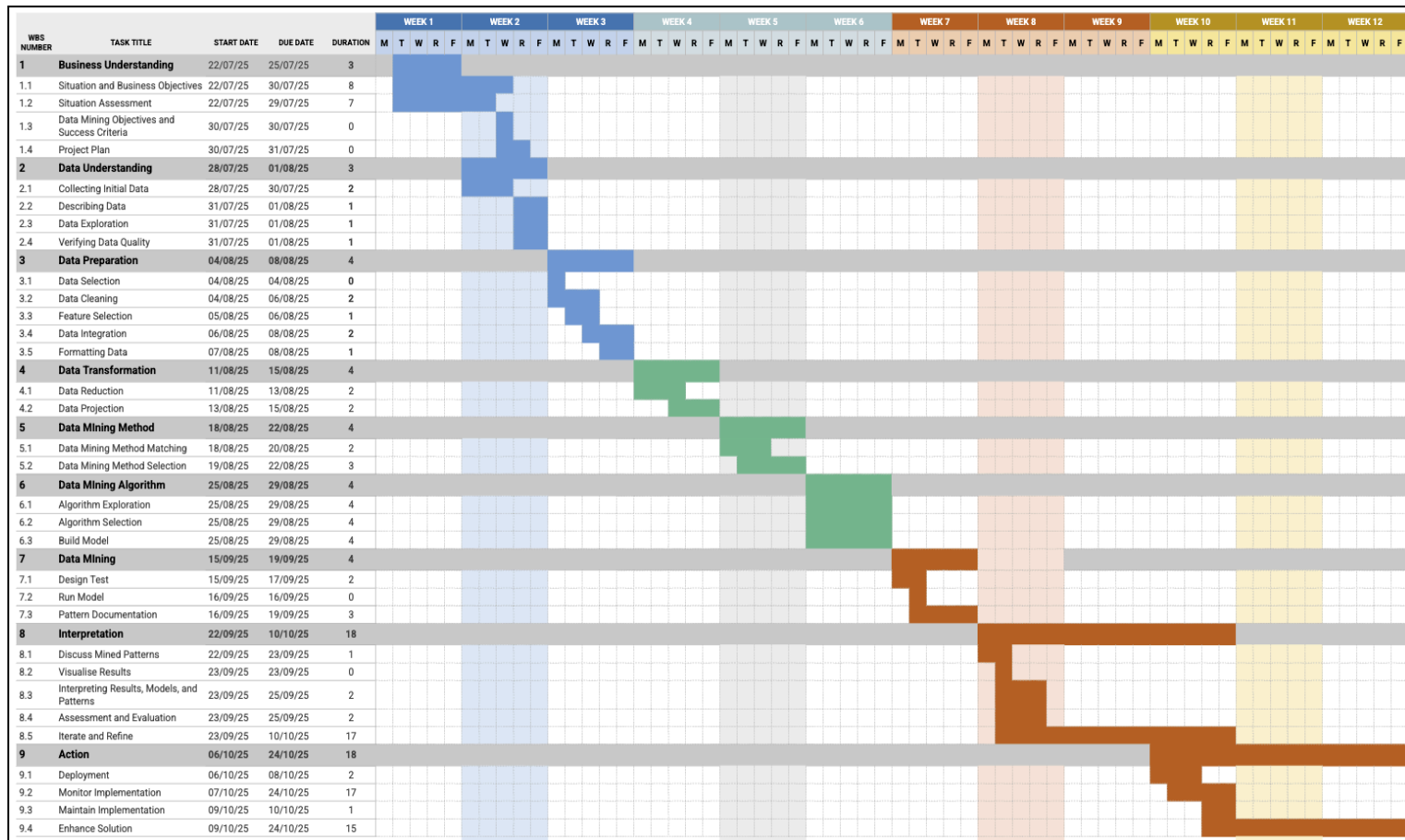
1.4. Project Plan

The project will follow the nine steps in data mining introduced in the course. Each step will be executed as shown in Table 1. The Gantt chart will be shown in Figure 1 to show the in-depth project timeline.

Phase	Allocated time in percentages	Iterations	Potential Risks
Business Understanding (BU)	10%	Multiple iterations could occur in these phases based on feedback.	Unclear project goals due to insufficient understanding of the domain.
Data Understanding (DU)	12.5%		The researcher could make incorrect assumptions.
Data Preparation (DP)	12.5%	Multiple iterations could occur in these phases based on the model output.	Missing data and outliers might be mishandled and could affect model training accuracy.
Data Transformation (DT)	12.5%		Over-transformation may remove meaningful patterns in the data.
Data Mining Model (DMM)	10%		Selecting an unsuitable method could lead to multiple iterations.

Data Mining Algorithm (DMA)	12.5%		Limited knowledge in model selection may result in the use of algorithms that are not optimal for the features selected.
Data Mining (DM)	12.5%		The model's performance may be poor due to an incorrect data mining technique from previous phases.
Interpretation	10%	This phase could go back to phase 1 depending on the output of the model.	The model outputs may be uninterpretable and clinically irrelevant for clinicians to understand the risk factors.
Action	7.5%		Unclear results or failure to justify the model decisions may compromise the project's practical value.

Table 1: Project Plan



2. Data Understanding (DU)

The dataset mentioned above was previously merged from the previous iteration with the [Dietary and Lifestyle dataset](#), which consists of synthetic patient records that focus on lifestyle and nutritional characteristics relevant to early detection. The purpose of this approach is to determine if combining complementary data sources improves predictive performance and provides a more solid foundation for survival modelling. However, it did not result positively and predictive performance did not improve beyond 60%. This leads the researcher to conclude that merging with the [Dietary and Lifestyle dataset](#) is ineffective and does not improve predictive performance and survival modelling.

For iteration three, the researcher will only work with the [Colorectal Cancer Global Dataset & Predictions](#) (Panday, 2025). Further adjustments will be applied to the chosen dataset to satisfy the next steps of data mining.

2.1. Collecting Initial Data

The course materials provided multiple sources for acquiring datasets, including government open data portals and public data science repositories. Among these, Kaggle provided the most comprehensive sources that met the research objectives. Three prospective datasets for consideration were identified through Kaggle.

The [Colorectal Cancer Global Dataset & Predictions](#) (Panday, 2025) was taken from Kaggle and contains medical records that capture factors impacting an individual's likelihood of survival for five years after CRC diagnosis. These factors include demographic characteristics, lifestyle and behavioural variables, medical history, and cancer-specific indicators. Furthermore, the dataset includes population health metrics as well as healthcare access variables, which could provide a more comprehensive context for assessing survival outcomes.

Other datasets were examined, such as the [Colorectal Cancer WSI dataset](#). It contains histological images for CRC diagnosis and can be applied for medical image classification. Another dataset considered is the [Dietary and Lifestyle dataset](#). It consists of synthetic patient records that focus on lifestyle and nutritional characteristics relevant to early detection. Neither dataset fits the data mining objectives and the target variables did not relate to patient survival for five years.

As for the [Colorectal Cancer Global Dataset & Predictions](#) (Panday, 2025) dataset, the variables can be divided into two categories: individual-level and contextual-level predictors. Individual-level predictors include age, gender, cancer stage, tumour size, family history, genetic mutation, diabetes, inflammatory bowel disease, smoking history, alcohol consumption, obesity, dietary risk, and physical activity. These variables are frequently identified in the literature as important predictors of colorectal cancer survival because they directly represent disease biology, patient health status, and modifiable risk factors that influence prognosis (Han et al., 2024; Maawadh et al., 2024). Screening history, early detection, and treatment type were also included, as research demonstrates that patients diagnosed early and treated effectively have considerably better five-year survival rates (Han et al., 2024; Heisser, 2024).

Contextual predictors such as urban or rural residence and insurance status were retained to account for inequalities in health outcomes caused by system-level and socioeconomic factors. According to studies, patients in rural areas or from lower socioeconomic backgrounds frequently experience challenges to timely diagnosis and effective treatment, resulting in lower survival outcomes (Fu et al., 2024; Klimeck et al., 2023; Sharples et al., 2018). Including these contextual variables allows the model to capture not only biological drivers of survival but also how systemic disparities influence patient outcomes. Collectively, this set of inputs offers a thorough depiction of both clinical and systemic determinants of colorectal cancer survival, which aligns with the research goal of predicting five-year survival while acknowledging health inequities.

Index	Patient_ID	Country	Age	Gender	Cancer_Stage	Tumor_Size_mm	Family_History	Smoking_History	Alcohol_Consumption	Obesity_BMI	Diet_Risk	Physical_Activity	Diabetes
0	1	UK	77	M	Localized	69	No	No	Yes	Overweight	Low	Low	No
1	2	UK	59	M	Localized	33	No	No	No	Overweight	Moderate	Low	No
2	3	Japan	66	M	Regional	17	No	Yes	No	Normal	Low	Low	No
3	4	USA	83	M	Regional	14	No	No	No	Obese	High	Low	No
4	5	France	66	M	Localized	34	No	Yes	No	Normal	Low	Moderate	No
5	6	Japan	69	M	Localized	71	No	No	Yes	Obese	Moderate	Low	Yes

Figure 2: Colorectal Cancer dataset excluded variables

Index	Diabetes	Inflammatory_Bowel_Disease	Genetic_Mutation	Screening_History	Early_Detection	Treatment_Type	Survival_5_years	Mortality	Healthcare_Costs	Incidence_Rate_per_100k
0	No	No	No	Regular	Yes	Combination	Yes	No	54413	50
1	No	No	No	Regular	No	Chemotherapy	Yes	No	76553	37
2	No	Yes	No	Irregular	No	Chemotherapy	Yes	No	62805	54
3	No	No	No	Regular	No	Surgery	Yes	No	89393	45
4	Yes	Yes	No	Never	Yes	Surgery	Yes	No	66425	15
5	Yes	Yes	No	Regular	No	Chemotherapy	Yes	No	89393	45

Figure 3: Colorectal Cancer dataset excluded variables with the target variable

Index	Mortality	Healthcare_Costs	Incidence_Rate_per_100k	Mortality_Rate_per_100k	Urban_or_Rural	Economic_Classification	Healthcare_Access	Insurance_Status	Survival_Prediction
0	No	54413	50	5	Urban	Developed	Moderate	Insured	Yes
1	No	76553	37	25	Urban	Developing	High	Uninsured	Yes
2	No	62805	54	27	Urban	Developed	Moderate	Uninsured	No
3	No	89393	45	11	Urban	Developed	Moderate	Insured	Yes
4	No	66425	15	27	Urban	Developing	High	Insured	Yes
5	No	89393	45	11	Urban	Developed	Moderate	Uninsured	No

Figure 4: Colorectal Cancer dataset excluded variables 2

Several variables from the original dataset were omitted from the model because they did not meet the research objectives of predicting five-year colorectal cancer survival based on individual and contextual characteristics. Patient_ID is eliminated since it serves only as a unique key with no analytical significance. Country-level attributes such as Country, Economic_Classification, Healthcare_Access, Healthcare_Costs, Incidence_Rate_per_100k, and Mortality_Rate_per_100k were also excluded since these variables represent aggregated population-level indicators rather than patient-specific predictors of survival. Similarly, outcome-related variables such as Mortality and Survival_Prediction were excluded because they either duplicated the target variable Survival_5_years or represented post-diagnostic outcomes that could result in data leakage. Excluding these variables meant that the variables chosen concentrated on individual-level clinical, lifestyle, and contextual predictors of colorectal cancer survival with empirical support in the literature (Han et al., 2024; Fu et al., 2024; Klimeck et al., 2023).

2.2. Describing Data

The [Colorectal Cancer Global Dataset & Predictions](#) (Panday, 2025) dataset is a structured, tabular dataset in comma-separated values (CSV) format. It contains 167,497 records and 28 fields showing an individual case relevant to CRC risk assessment and prediction.


```

In [5]:
...: print("Shape of dataset: ", df1.shape)
Shape of dataset: (167497, 28)

In [6]:
...: df1.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 167497 entries, 0 to 167496
Data columns (total 28 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Patient_ID                               167497 non-null  int64
1   Country                                  167497 non-null  object
2   Age                                       167497 non-null  int64
3   Gender                                   167497 non-null  object
4   Cancer_Stage                             167497 non-null  object
5   Tumor_Size_mm                           167497 non-null  int64
6   Family_History                           167497 non-null  object
7   Smoking_History                          167497 non-null  object
8   Alcohol_Consumption                     167497 non-null  object
9   Obesity_BMI                             167497 non-null  object
10  Diet_Risk                                167497 non-null  object
11  Physical_Activity                        167497 non-null  object
12  Diabetes                                  167497 non-null  object
13  Inflammatory_Bowel_Disease               167497 non-null  object
14  Genetic_Mutation                         167497 non-null  object
15  Screening_History                        167497 non-null  object
16  Early_Detection                          167497 non-null  object
17  Treatment_Type                           167497 non-null  object
18  Survival_5_years                         167497 non-null  object
19  Mortality                                167497 non-null  object
20  Healthcare_Costs                         167497 non-null  int64
21  Incidence_Rate_per_100K                  167497 non-null  int64
22  Mortality_Rate_per_100K                  167497 non-null  int64
23  Urban_or_Rural                           167497 non-null  object
24  Economic_Classification                  167497 non-null  object
25  Healthcare_Access                        167497 non-null  object
26  Insurance_Status                         167497 non-null  object
27  Survival_Prediction                      167497 non-null  object
dtypes: int64(6), object(22)
memory usage: 35.8+ MB

```

Figure 5: Screenshot of the summary of the DataFrame's key characteristics (refer to [Appendix B](#) for full code snippet)

The dataset contains a diverse set of fields. It can be split into different categories:

- Demographic
 - Age, Gender, and Country
- Lifestyle and behavioural risks
 - Smoking_History, Alcohol_Consumption, Diet_Risk, and Obesity_BMI
- Medical history
 - Family_History, Diabetes and Inflammatory_Bowel_Disease
- Cancer-specific indicators
 - Cancer_Stage and Tumor_Size_mm, Survival_5_years, and Survival_Prediction
- Population health
 - Incidence_Rate_per_100k and Mortality_Rate_per_100k
- Healthcare access
 - Healthcare_Costs and Healthcare_Access

Each row can be identified by a unique Patient_ID. The current dataset displays a non-null count of 167,497 for all fields. All fields are identified as object types except for fields such as Patient_ID, Age, Tumor_Size_mm, Healthcare_Costs, Incidence_Rate_per_100k and Mortality_Rate_per_100k, as these values contain numerical values. The rest of the fields are object type because this is how pandas works with text data (Working With Text Data — Pandas 2.3.2 Documentation, n.d.).

2.3. Data Exploration

This section aims to identify underlying patterns in the dataset by examining the distributions of individual variables and the relationships between them.

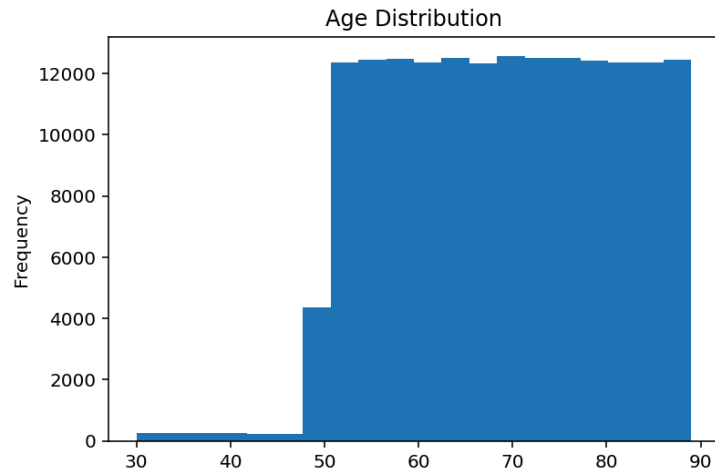


Figure 6: Screenshot for age distribution

An age distribution of the dataset reveals that the majority of the records show an increase in CRC cases within the 50 to 90 age range. However, there is a decrease in cases for individuals below 50. The data suggest that the disease predominantly develops in older adults between 50 and 90 years old, with few records showing it in the late 40s age range. The fall of records below age 45 may indicate underrepresentation of younger individuals in the dataset or a possible age-based inclusion criterion.

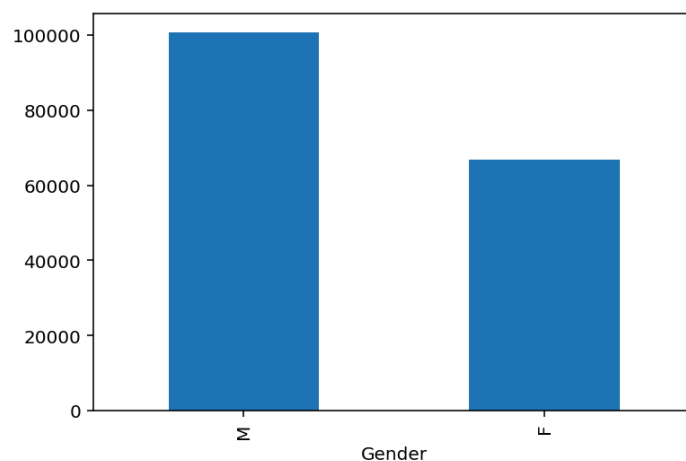


Figure 7: Screenshot for gender distribution

A visualisation of the gender distribution across all datasets reveals that more males than females are diagnosed with CRC.

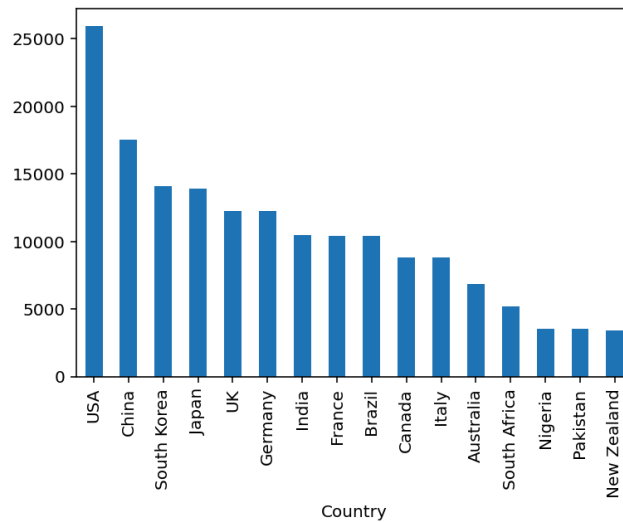


Figure 8: Screenshot for country distribution

The dataset contains CRC cases from countries including the USA, China, South Korea, Japan, the UK, Germany, India, France, Brazil, Canada, Italy, Australia, South Africa, Nigeria, Pakistan, and New Zealand. More CRC cases come from the USA than from any other country. New Zealand has the fewest CRC cases within the dataset.

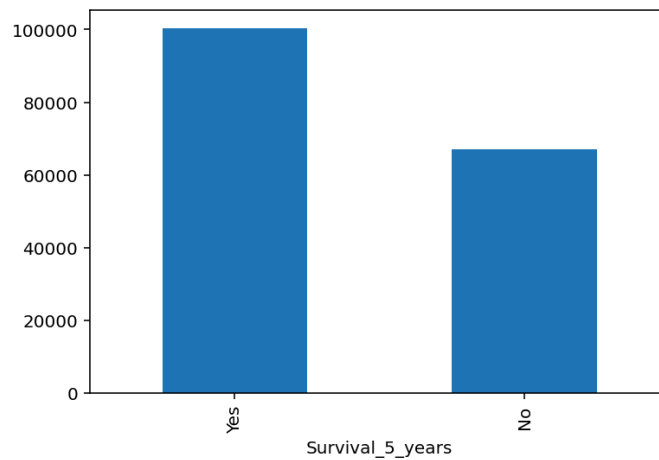


Figure 9: Screenshot for patient five-year survival distribution

Visualising the distribution of patient survival across five years may reveal that the majority of cases survived five years following diagnosis. The distribution shows that 100,387 instances survived five years following diagnosis, while 67,110 did not.

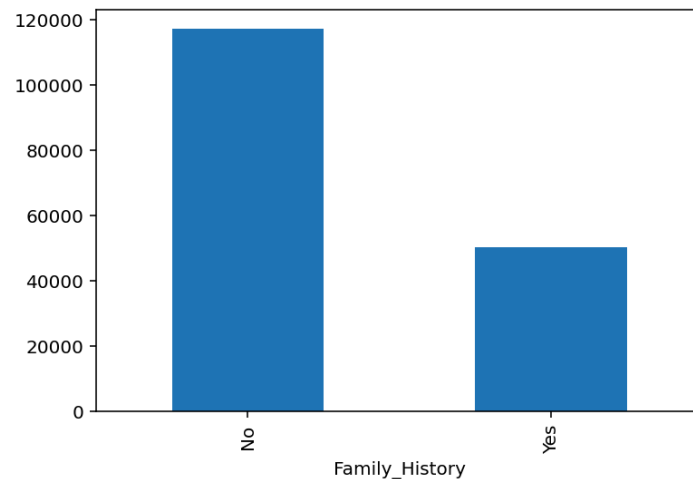


Figure 10: Screenshot for CRC family history distribution

Patients without a known family history of CRC account for the majority of cases in the sample. Family history is a key predictive factor that influences the survival outcomes of colorectal cancer (Bass et al., 2008). As a result, it can be concluded that the dataset's comparatively greater percentage of survivors may be partially explained by the prevalence of individuals without a family history of CRC which are typically linked to better outcomes for survival.

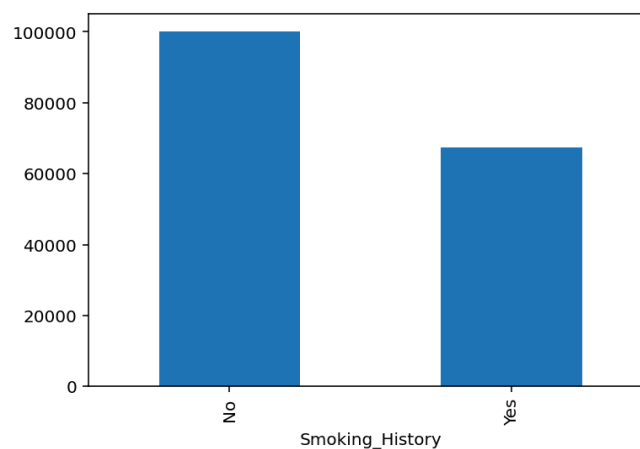


Figure 11: Screenshot for smoking history distribution

The distributions show that the majority of patients did not have a history of smoking. It is identified from numerous medical studies that smoking is a risk factor influencing CRC incidence and mortality (Sharp et al., 2017; Chao et al., 2000; Walter et al., 2014). Therefore, it may be said that the large proportion of individuals without a history of smoking may contribute to the dataset's relatively higher percentage of survivors.

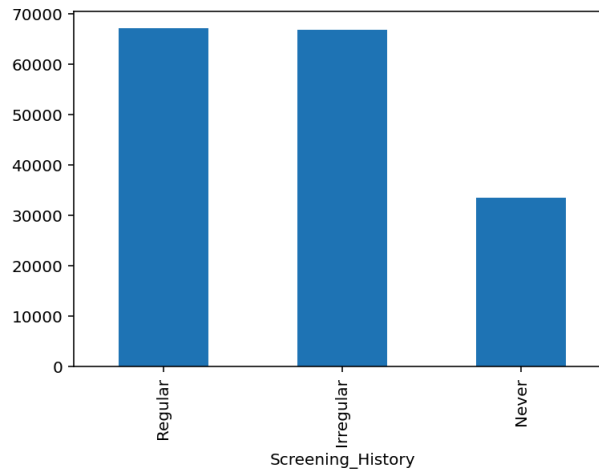


Figure 12: Screenshot for screening history distribution

The majority of patients had either regular or irregular screening for CRC. While a smaller number of patients had never been screened. Medical research shows that CRC identified by population-based screening programmes is linked to a much greater survival and lower rate of tumour recurrence (Mengual-Ballester et al., 2016).

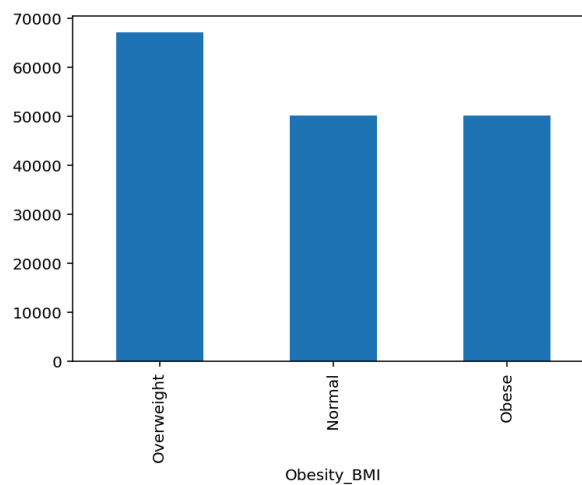


Figure 13: Screenshot for BMI distribution

Body Mass Index (BMI) values at normal, overweight, and obese levels are displayed in the bar chart. According to the distributions, the majority of patients are overweight. The remainder of the distribution is composed of nearly equal numbers of normal and obese patients.

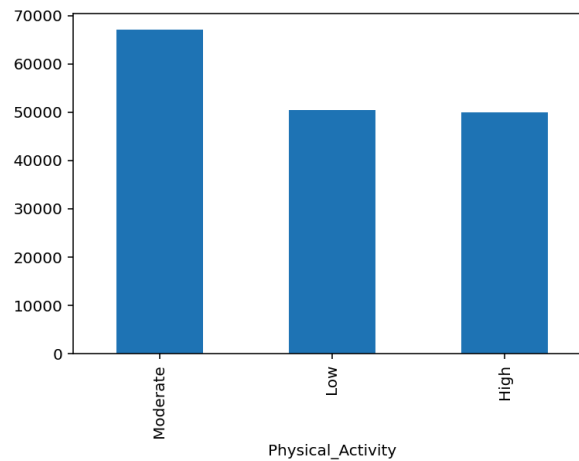


Figure 14: Screenshot for physical activity distribution

The distribution of patients in the dataset across low, moderate, and high levels of physical activity is depicted in the bar chart. While the numbers of patients in the low and high activity categories were about equal, the largest group indicated moderate physical activity.

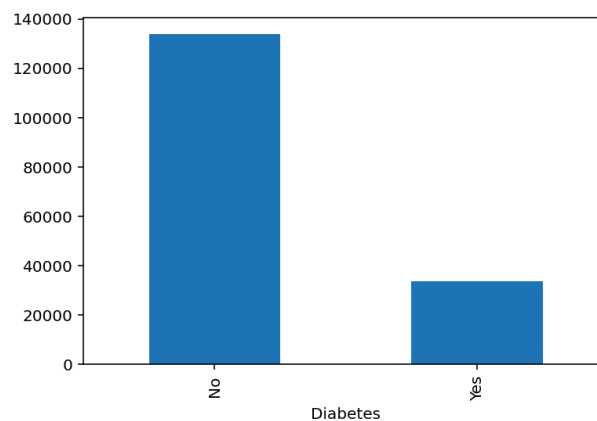


Figure 15: Screenshot for diabetes distribution

The bar chart shows the distribution of diabetic patients after CRC diagnosis. The majority of patients do not have diabetes upon CRC diagnosis. Medical studies show that diabetes influences CRC patients' five-year survival (Zhu et al., 2017). Consequently, it might be argued that the high percentage of individuals without diabetes may be a factor in the dataset's comparatively high survival rate.

2.4. Verifying Data Quality

This section involves inspecting the dataset to identify errors, missing values, and irregular data patterns. Verifying the data is essential for the reliability and validity of the data mining outcomes, particularly in contexts where the results may inform clinical decision-making processes.

```

In [10]: missing_per_col = df1.isnull().sum()
...: print(df1.isnull().sum())
Patient_ID      0
Country         0
Age             0
Gender          0
Cancer_Stage    0
Tumor_Size_mm   0
Family_History  0
Smoking_History 0
Alcohol_Consumption 0
Obesity_BMI     0
Diet_Risk       0
Physical_Activity 0
Diabetes        0
Inflammatory_Bowel_Disease 0
Genetic_Mutation 0
Screening_History 0
Early_Detection 0
Treatment_Type  0
Survival_5_years 0
Mortality       0
Healthcare_Costs 0
Incidence_Rate_per_100K 0
Mortality_Rate_per_100K 0
Urban_or_Rural  0
Economic_Classification 0
Healthcare_Access 0
Insurance_Status 0
Survival_Prediction 0
dtype: int64

```

Figure 16: Screenshot of verifying null values in the dataset
(refer to [Appendix C](#) for full code snippet)

The data audit confirms that all of the datasets exhibit complete integrity with 100% completeness across all fields and no evidence of null values, empty strings, white spaces, or black values.

```

In [7]: print("Duplicate rows:", df1.duplicated().sum())
Duplicate rows: 0

In [8]: def iqr_outliers(series):
...:     Q1 = series.quantile(0.25)
...:     Q3 = series.quantile(0.75)
...:     IQR = Q3 - Q1
...:     lower_bound = Q1 - 1.5 * IQR
...:     upper_bound = Q3 + 1.5 * IQR
...:     outliers = series[(series < lower_bound) | (series > upper_bound)]
...:     return outliers, lower_bound, upper_bound
...:
...: # outliers in Age
...: outlier_age, lower, upper = iqr_outliers(df1["Age"])
...: print(f"Outliers in Age: {len(outlier_age)} values")
...: print(f"Lower bound: {lower}, Upper bound: {upper}")
...:
...: outlier_tumor_size, lower, upper = iqr_outliers(df1["Tumor_Size_mm"])
...: print(f"Outliers in Tumor_Size_mm: {len(outlier_tumor_size)} values")
...: print(f"Lower bound: {lower}, Upper bound: {upper}")
Outliers in Age: 0 values
Lower bound: 29.0, Upper bound: 109.0
Outliers in Tumor_Size_mm: 0 values
Lower bound: -34.0, Upper bound: 118.0

```

Figure 17: Screenshot of verifying duplicate and outlier values in the dataset
(refer to [Appendix D](#) for full code snippet)

Age and Tumor_Size_mm did not contain any outliers. The age range for the threshold was 29-109 years old. For Tumor_Size_mm, the top and lower thresholds were 118 mm and -34 mm, respectively. There were no negative values in the dataset. Tumour sizes cannot be negative. Every tumour size value fell into a legitimate clinical range. Additionally, Figure 9 shows that there is an imbalance in the distribution of the target variable Survival_5_years. As a result, the researcher concludes that the dataset is clean as of this step.

3. Data Preparation (DP)

This section involves an important step in the knowledge discovery process. During this phase, the dataset is properly refined for further modelling. This section summarises significant preparation operations carried out to improve the dataset's analytical applicability. Selecting relevant variables based on objectives, data quality assessments, identifying and resolving data inconsistencies, developing new features to improve predictive capacity, integrating data sources to ensure coherence, and reformatting data types to meet modelling requirements.

3.1. Data Selection

Index	Patient_ID	Age	Gender	Cancer_Stage	Tumor_Size_mm	Family_History	Smoking_History	Alcohol_Consumption	Obesity_BMI	Diet_Risk	Physical_Activity	Diabetes	Inflammatory_Bov
0	1	77	M	nan	69	No	No	Yes	Overweig...	Low	Low	No	No
1	2	59	M	Localized	33	No	No	No	Overweig...	Modera...	Low	No	No
2	3	66	M	nan	nan	No	Yes	No	Normal	Low	Low	No	Yes
3	4	83	M	Regional	14	No	No	No	Obese	High	Low	No	No
4	5	66	M	Localized	34	No	Yes	No	Normal	Low	Moderate	No	Yes
5	6	60	M	Localized	71	No	No	Yes	Obese	Moderate	Low	Yes	Yes

Index	Inflammatory_Bowel_Disease	Genetic_Mutation	Screening_History	Early_Detection	Treatment_Type	Survival_5_years	Healthcare_Costs	Urban_or_Rural	Healthcare_Access	Insurance
0	No	No	Regular	Yes	Combination	Yes	54413	Urban	Moderate	Insured
1	No	No	Regular	No	Chemotherapy	Yes	76553	Urban	High	Uninsured
2	Yes	No	Irregular	No	Chemotherapy	Yes	62805	Urban	Moderate	Uninsured
3	No	No	Regular	No	Surgery	Yes	89393	Urban	Moderate	Insured
4	Yes	No	Never	Yes	Surgery	Yes	66425	Urban	High	Insured
5	Yes	No	Regular	No	Radiotherapy	No	82764	Urban	Moderate	Uninsured

Index	Inflammatory_Bowel_Disease	Genetic_Mutation	Screening_History	Early_Detection	Treatment_Type	Survival_5_years	Healthcare_Costs	Urban_or_Rural	Healthcare_Access	Insurance_Status
0		No	Regular	Yes	Combination	Yes	54413	Urban	Moderate	Insured
1		No	Regular	No	Chemotherapy	Yes	76553	Urban	High	Uninsured
2		No	Irregular	No	Chemotherapy	Yes	62805	Urban	Moderate	Uninsured
3		No	Regular	No	Surgery	Yes	89393	Urban	Moderate	Insured
4		No	Never	Yes	Surgery	Yes	66425	Urban	High	Insured
5		No	Regular	No	Radiotherapy	No	82764	Urban	Moderate	Uninsured

Figure 18: Screenshot of selected columns

Variables were chosen based on their theoretical and empirical importance to colorectal cancer survival, as validated by current research (Han et al., 2024; Maawadh et al., 2024; Heisser, 2024). To capture biological, behavioural, and systemic factors of survival, predictors at both the individual and contextual levels were kept. This included demographic variables (age and gender), lifestyle and behavioural risk factors (smoking history, alcohol consumption, diet risk, physical activity, obesity/body mass index), medical history (family history of colorectal cancer, diabetes, hypertension, inflammatory bowel disease, genetic mutation), clinical factors (cancer stage, tumour size, treatment type, early detection, screening history), and contextual characteristics (urban or rural residence).

Name	Type	Size	Value
Altered_Dataset	str	37	colorectal_cancer_dataset_altered.c...
col	str	17	Physical_Activity
cols_with_nulls	list	5	['Age', 'Tumor_Size_mm', 'Cancer_St...
df1	DataFrame	[167497, 28]	Column names: Patient_ID, Country, ...
df1_altered	DataFrame	[167497, 28]	Column names: Patient_ID, Country, ...
df_selected	DataFrame	[167497, 22]	Column names: Patient_ID, Age, Gend...
dfa	DataFrame	[167497, 28]	Column names: Patient_ID, Country, ...
drop_cols	list	6	['Country', 'Mortality', 'Incidence...
keep_cols	list	22	['Patient_ID', 'Age', 'Gender', 'Ca...
KEY	str	10	Patient_ID
null_indices	Index	[8375]	Index: 8375 entries, 96053 to 107333
Primary_Dataset	str	29	colorectal_cancer_dataset.csv
TARGET	str	16	Survival_5_years

Figure 19: Screenshot of the data frame before and after data selection

The target variable selected is Survival_5_years. Columns such as Country, Mortality, Incidence_Rate_per_100K, Mortality_Rate_per_100K, Economic_Classification, and Survival_Prediction have been removed as these variables did not represent individual-level predictors and were irrelevant to the research objectives. During the data selection process, Patient_ID is retained because it will be essential for the subsequent stage of dividing the data into multiple smaller tables. Figure 18 shows how the columns decreased from 28 columns to 22 after data selection. The researcher also added null values to the columns for Age, Tumor_Size_mm, Cancer_Stage, Obesity_BMI, and Physical_Activity in order to meet the requirements of the phases that follow.

3.2. Data Cleaning

```

print('No duplicates found (all var...')
Missing values per column before imputation:
Age          8375
Tumor_Size_mm  8375
Cancer_Stage  8375
Obesity_BMI   8375
Physical_Activity  8375
dtype: int64
Selected shape: (167497, 22)
Imputed Age with median value 69.0
Imputed Tumor_Size_mm with median value 42.0
Imputed Cancer_Stage with mode value 'Regional'
Imputed Obesity_BMI with mode value 'Overweight'
Imputed Physical_Activity with mode value 'Moderate'

```

Figure 20: Screenshot of missing values before imputation
(refer to [Appendix E](#) for full code snippet)

Each of the chosen variables (Age, Tumor_Size_mm, Cancer_Stage, Obesity_BMI, and Physical_Activity) had 8,375 missing values before cleaning the data. In order to apply the data cleaning process, these missing values were purposefully added as part of the data preparation process, regardless of whether they were not initially present in the dataset.

```

Missing values per column AFTER imputation:
Age                0
Tumor_Size_mm     0
Cancer_Stage       0
Obesity_BMI        0
Physical_Activity  0
dtype: int64
Selected shape: (167497, 22)
Gender: no space issues detected
Cancer_Stage: no space issues detected
Family_History: no space issues detected
Smoking_History: no space issues detected
Alcohol_Consumption: no space issues detected
Obesity_BMI: no space issues detected
Diet_Risk: no space issues detected
Physical_Activity: no space issues detected
Diabetes: no space issues detected
Inflammatory_Bowel_Disease: no space issues detected
Genetic_Mutation: no space issues detected
Screening_History: no space issues detected
Early_Detection: no space issues detected
Treatment_Type: no space issues detected
Survival_5_years: no space issues detected
Urban_or_Rural: no space issues detected
Healthcare_Access: no space issues detected
Insurance_Status: no space issues detected

Column: Patient_ID
No duplicates found (all values unique).

```

Figure 21: Screenshot of missing values before imputation
(refer to [Appendix E](#) for full code snippet)

Null values were addressed according to variable type. Categorical variables were imputed with their mode values (*Cancer_Stage* = Regional, *Obesity_BMI* = Overweight, *Physical_Activity* = Moderate). Continuous variables were imputed with their median values (*Age* = 69.0, *Tumor_Size_mm* = 42.0). Additional checks were conducted to detect leading or trailing white spaces in text fields and to identify potential duplication of *Patient_ID* before proceeding to the next step.

3.3. Data Construct

Name	Type	Size	Value
patient_access	DataFrame	[167497, 6]	Column names: Patient_ID, Heal
patient_clinical	DataFrame	[167497, 7]	Column names: Patient_ID, Canc
patient_costs	DataFrame	[167497, 2]	Column names: Patient_ID, Heal
patient_demographics	DataFrame	[167497, 3]	Column names: Patient_ID, Age,
patient_lifestyle	DataFrame	[167497, 6]	Column names: Patient_ID, Smok
patient_outcomes	DataFrame	[167497, 2]	Column names: Patient_ID, Surv
patient_treatment	DataFrame	[167497, 4]	Column names: Patient_ID, Trea

Figure 22: Screenshot of the divided dataset
(refer to [Appendix F](#) for full code snippet)

The dataset is split into several tables:

- Patient_demographics: Patient_ID, Age, and Gender
- Patient_clinical: Patient_ID, Cancer_Stage, Tumor_Size_mm, Family_History, Diabetes, Inflammatory_Bowel_Disease, and Genetic_Mutation
- Patient_lifestyle: Patient_ID, Smoking_History, Alcohol_Consumption, Obesity_BMI, Diet_Risk, and Physical_Activity
- Patient_treatment: Patient_ID, Treatment_Type, Screening_History, and Early_Detection
- Patient_access: Patient_ID, Healthcare_Access, insured, and uninsured
- Patient_costs: Patient_ID and Healthcare_Costs
- Patient_outcomes: Patient_ID and Survival_5_years

The Patient_ID field was kept consistent across all tables to allow merging in later stages. The dataset was divided into distinct tables that represented different components of patient data in order to mimic the structure of an electronic health record (EHR). The insured and uninsured variables in the patient_access table were the result of the first one-hot encoding process used to prepare the data.

3.4. Data Integration

```
...: "urban", "rural", "Healthcare_Access",
...: "insured", "uninsured"
...: ]
...:
...: df_integrated = df_integrated[final_columns].copy()
...:
...: # df_selected = merged tables with one-hot encoding = 167497 rows, 24 col
...: # df_selected = patient-specific columns only = 167497 rows, 22 col
...: # df1 = original dataset = 167497 rows, 28 col
...: print("Integrated dataset shape:", df_integrated.shape)
...: print("Selected dataset shape: ", df_selected.shape)
...: print("Original dataset shape: ", df1.shape)
...:
...: # checks if merged properly
...: n_rows = len(df_integrated)
...: n_unique_ids = df_integrated["Patient_ID"].nunique()
...: print(f"Rows in final dataset: {n_rows}")
...: print(f"Unique Patient_IDs: {n_unique_ids}")
...:
...: if n_rows == n_unique_ids:
...:     print("No duplicate Patient_IDs")
...: else:
...:     print("Duplicate found")
...:
...: missing_ids = df_integrated["Patient_ID"].isna().sum()
...: print(f"Missing Patient_IDs: {missing_ids}")
...:
...: # check if there's still null values
...: total_nulls = df_integrated.isnull().sum().sum()
...: print(f"Total null values in dataset: {total_nulls}")
Integrated dataset shape: (167497, 24)
Selected dataset shape: (167497, 22)
Original dataset shape: (167497, 28)
Rows in final dataset: 167497
Unique Patient_IDs: 167497
No duplicate Patient_IDs
Missing Patient_IDs: 0
Total null values in dataset: 0
```

Figure 23: Screenshot of data integration
(refer to [Appendix G](#) for full code snippet)

The datasets were combined into a single table using Patient_ID. After merging, the columns were rearranged to match the original dataset order. Checks were done to ensure that Patient_ID values were unique, without duplicates, missing entries, or nulls. The dataset at this point included two additional columns created from the one-hot encoding of the Urban_or_Rural variable.

3.5. Data Reformatting

Indi	Age	Gender_F	Gender_M	Cancer_Stage_Localized	Cancer_Stage_Metastatic	Cancer_Stage_Regional	Tumor_mm	Family_History	Smoking_History	Alcohol_Consumption	BMI_Norm
0	77	0	1	0	0	1	69	False	False	True	0
1	59	0	1	1	0	0	33	False	False	False	0
2	66	0	1	0	0	1	42	False	True	False	1
3	83	0	1	0	0	1	14	False	False	False	0
4	66	0	1	1	0	0	34	False	True	False	1
5	60	0	1	1	0	0	71	False	False	True	0

Indi	BMI_Normal	BMI_Obese	BMI_Overweight	Diet_Risk	Physical_Activity	Diabetes	IBD	Genetic_Mutation	Screening_History_Irregular	Screening_History_Never	Screening_History_Regular
0	0	0	1	0	0	False	False	False	0	0	1
1	0	0	1	1	0	False	False	False	0	0	1
2	1	0	0	0	0	False	True	False	1	0	0
3	0	1	0	2	0	False	False	False	0	0	1
4	1	0	0	0	1	False	True	False	0	1	0
5	0	1	0	1	0	True	True	False	0	0	1

Indi	Screening_History_Regular	Early_Detection	Treatment_Type_Chemotherapy	Treatment_Type_Combination	Treatment_Type_Radiotherapy	Treatment_Type_Surgery	Survival_5_yrs
0	1	True	0	1	0	0	True
1	1	False	1	0	0	0	True
2	0	False	1	0	0	0	True
3	1	False	0	0	0	1	True
4	0	True	0	0	0	1	True
5	1	False	0	0	1	0	False

Indi	Treatment_Type_Combination	Treatment_Type_Radiotherapy	Treatment_Type_Surgery	Survival_5_yrs	HC_Costs	Urban	Rural	Healthcare_Access	Insured	Uninsured
0	1	0	0	True	54413	1	0	1	1	0
1	0	0	0	True	76553	1	0	2	0	1
2	0	0	0	True	62805	1	0	1	0	1
3	0	1	1	True	89393	1	0	1	1	0
4	0	1	1	True	66425	1	0	2	1	0
5	0	1	0	False	82364	1	0	1	0	1

Figure 24: Screenshot of data reformatting
(refer to [Appendix H](#) for full code snippet)

Once the dataset has been merged, Patient_ID is removed from the dataset. The following columns are renamed for readability:

- Tumor_Size_mm → Tumor_mm
- Obesity_BMI → BMI
- Inflammatory_Bowel_Disease → IBD
- Survival_5_years → Survival_5_yrs
- Healthcare_Costs → HC_Costs
- urban → Urban
- rural → Rural
- insured → Insured
- uninsured → Uninsured

Family_History, Smoking_History, Alcohol_Consumption, Diabetes, IBD, Genetic_Mutation, Early_Detection, and Survival_5_yrs columns contain yes or no values. This is changed into True or False. One-hot encoding was used to transform the categorical variables, including Gender, Cancer_Stage, BMI, Screening_History, and Treatment_Type into dummy indicator columns. Ordinal variables such as Diet_Risk, Physical_Activity, and Healthcare_Access were converted to ordered integers. The values within these columns were converted into the following:

- Low → 0
- Moderate → 1
- High → 2

```

Final types: Age                               Int64
Gender_F                                         Int8
Gender_M                                         Int8
Cancer_Stage_Localized                         Int8
Cancer_Stage_Metastatic                       Int8
Cancer_Stage_Regional                         Int8
Tumor_mm                                       Int64
Family_History                                boolean
Smoking_History                              boolean
Alcohol_Consumption                          boolean
BMI_Normal                                    Int8
BMI_Obese                                     Int8
BMI_Overweight                               Int8
Diet_Risk                                     Int8
Physical_Activity                             Int8
Diabetes                                       boolean
IBD                                           boolean
Genetic_Mutation                             boolean
Screening_History_Irregular                   Int8
Screening_History_Never                      Int8
Screening_History_Regular                     Int8
Early_Detection                              boolean
Treatment_Type_Chemotherapy                   Int8
Treatment_Type_Combination                     Int8
Treatment_Type_Radiotherapy                   Int8
Treatment_Type_Surgery                       Int8
Survival_5_yrs                               boolean
HC_Costs                                     Int64
Urban                                         Int8
Rural                                         Int8
Healthcare_Access                           Int8
Insured                                       Int8
Uninsured                                    Int8
dtype: object

```

Figure 25: Screenshot of setting dtype for all columns

The columns were rearranged to reflect the original dataset column order. A data type was assigned to each column in the dataset based on the type of information it stores. For instance:

- One-hot encoded columns contain 0 and 1 values: Int8
- Continuous variables: Int64
- Binary variables (yes/no columns turned into True/False): boolean

Data reformatting processes mentioned above are applied to provide consistency and standardisation to improve data quality, reduce noise, and prepare the dataset for machine learning algorithms. After data reformatting, the dataset included 167497 rows and 33 columns. The table was exported as an index-free CSV file.

4. Data Transformation (DT)

The transformation stage was used to optimise the dataset's performance in predictive modelling. This included both reduction and projection, ensuring that the final dataset was not only analytically sound but also computationally efficient.

4.1. Data Reduction

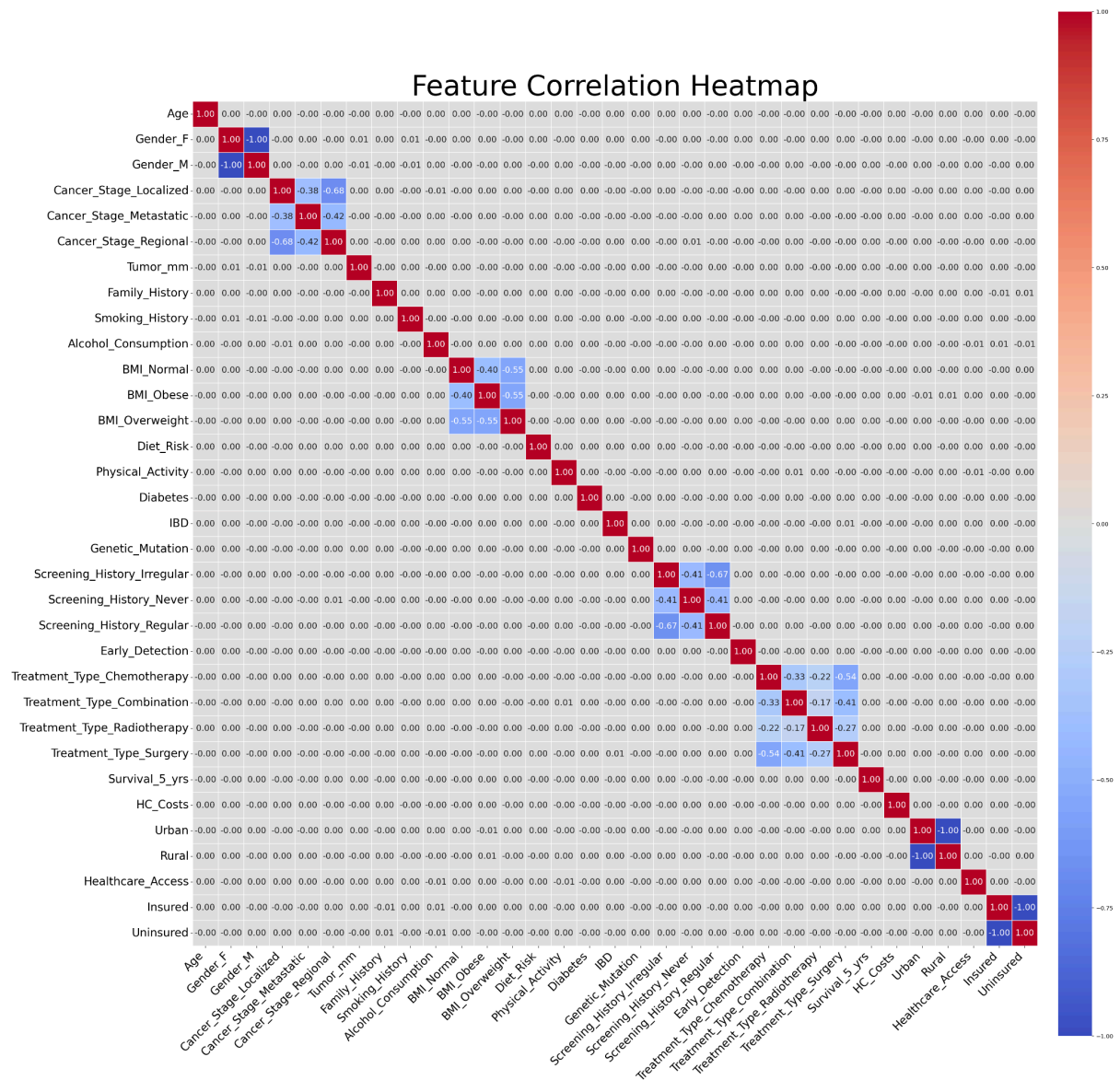


Figure 26: Correlation heatmap between columns
(refer to [Appendix I](#))

A correlation heatmap is produced to visualise how columns relate to each other. It can be seen that columns generated after applying one-hot encoding show high correlation towards each other. These columns include Gender_F, Gender_M, Cancer_Stage_Localized, Cancer_Stage_Metastatic, Cancer_Stage_Regional, BMI_Normal, BMI_Obese, BMI_Overweight, Screening_History_Irregular, Screening_History_Never, Screening_History_Regular, Treatment_Type_Chemotherapy, Treatment_Type_Combination, Treatment_Type_Radiotherapy, Treatment_Type_Surgery, Urban, Rural, Insured, and Uninsured. Columns generated from one-hot encoding need to be reduced.


```

....: print("Top MI features:", mi_features)
Top chi2 features: ['Family_History', 'Alcohol_Consumption', 'BMI_Normal',
'Screening_History_Irregular', 'Screening_History_Never', 'Treatment_Type_Surgery', 'Urban', 'Rural',
'Insured', 'Uninsured']
Top MI features: ['Gender_F', 'Gender_M', 'Smoking_History', 'Alcohol_Consumption', 'BMI_Overweight',
'Screening_History_Regular', 'Early_Detection', 'Treatment_Type_Surgery', 'Urban', 'Insured']

....: print("Final selected features:", final_features)
Final selected features: ['Age', 'Alcohol_Consumption', 'BMI_Obese', 'BMI_Overweight',
'Cancer_Stage_Metastatic', 'Cancer_Stage_Regional', 'Diabetes', 'Diet_Risk', 'Early_Detection',
'Family_History', 'Gender_F', 'Genetic_Mutation', 'IBD', 'Insured', 'Physical_Activity',
'Screening_History_Irregular', 'Screening_History_Never', 'Smoking_History',
'Treatment_Type_Chemotherapy', 'Treatment_Type_Radiotherapy', 'Treatment_Type_Surgery', 'Tumor_mm',
'Urban']

....: # checks
....: print("Original dataset shape:", df.shape) # 167497, 33
....: print("Reduced dataset shape:", df_reduced.shape) # 167497, 24
Original dataset shape: (167497, 33)
Reduced dataset shape: (167497, 24)

```

Figure 27: Reducing columns and final dataset
(refer to [Appendix I](#))

The researcher applied chi-squared to determine highly correlated categorical variables, mutual information (MI) for feature selection, and selection of features according to medical studies. Duplicated columns from one-hot encoding are dropped to create the final selected features. The dataset contained 167497 rows and 33 columns during data preparation. Data reduction resulted in fewer columns, making the current dataset have 167497 rows and 24 columns.

4.2. Data Projection

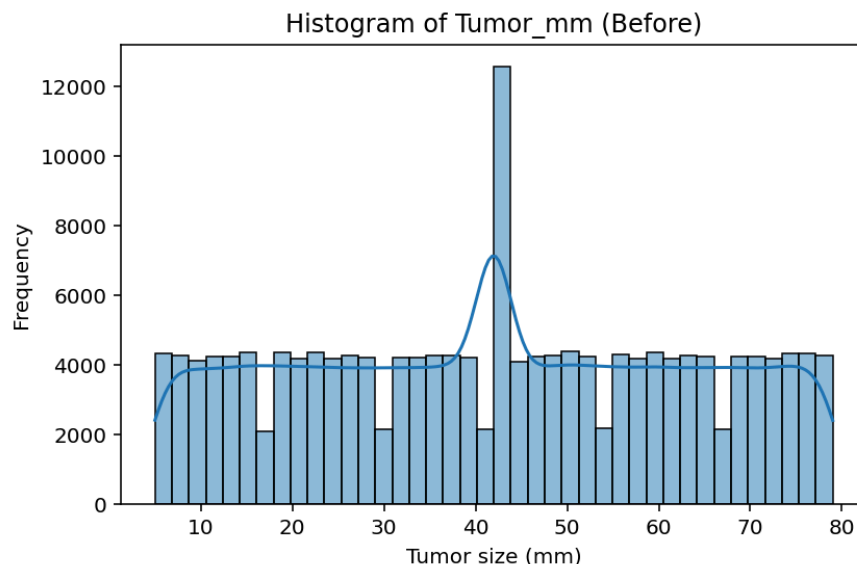


Figure 28: Screenshot of Tumor_mm

An almost flat histogram of Tumor_mm can be seen in Figure 26. The researcher attempted to apply the log, the square root, and the Box-Cox to transform the feature into a more symmetrical and normal-like shape.

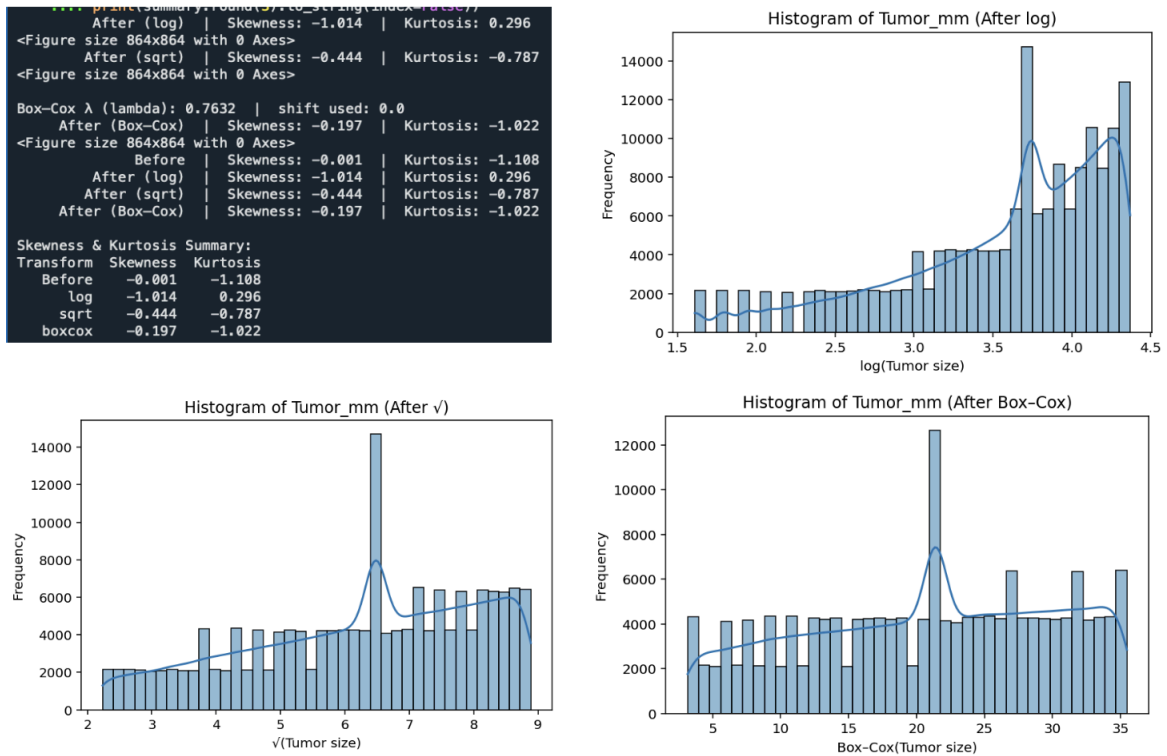


Figure 29: Screenshot of Tumor_mm after log, square root, and Box-Cox

The dataset documentation states that tumour size values were produced at random within medical limits (Panday, 2025). Testing data projection with Tumor_mm was still necessary to determine whether the feature might be reshaped into a more model-friendly distribution. At this point, two versions of the dataset were saved: one with Tumor_mm in its original form and another with the Box-Cox transformation applied. The Box-Cox transformation was chosen because it gave a distribution of Tumor_mm that was more balanced and closer to normal than the log and square root transformations. Retaining both datasets provides flexibility for future testing and comparison.

5. Data Mining Methods (DMM)

5.1. Data Mining Methods Context

The goal of this study is to predict five-year survival outcomes for patients with CRC, with Survival_5_yrs defined as a binary outcome. Given the categorical features of the dependent variable, supervised learning methods that specialise in classification are the most suitable choice. Classification algorithms are well established in the medical informatics literature for survival analysis because they can handle heterogeneous predictors, capture non-linear relationships, and produce interpretable results for clinical decision support (Han et al., 2024; Maawadh et al., 2024). For this reason, the study does not rely on a single model. Instead, it evaluates several classification algorithms. This method allows the researcher to assess accuracy, robustness, and interpretability across models. It also guarantees that the selected algorithm and model are based on evidence rather than assumptions. Although classification

does not provide a precise survival time, it is still the best option in this case since it provides unambiguous and reliable predictions that aid in clinical decisions.

5.2. Data Mining Method Selection

In this study, numerous categorisation approaches have been explored to identify which one best predicts five-year survival in CRC patients. Each strategy has a distinct way of extracting patterns from data. For example:

- Extreme Gradient Boosting (XGBoost): This method generates several small decision trees, with each new tree correcting errors in the previous ones. It has been demonstrated in research to be highly accurate in CRC survival prediction (Nopour, 2024).
- Random Survival Forest (RSF): This method generates a large number of survival trees to estimate the likelihood of surviving over a given time period, such as five years. It is notably beneficial for censored data and has been shown in clinical research to be a reliable survival analysis method (Yang et al., 2023).
- Naive Bayes: This is a simple and efficient way of making predictions based on probability. Despite the assumption of feature independence, it has been successfully used in CRC prediction studies (Mohammadi et al., 2024).

The three algorithms are selected due to their proven high accuracy in predicting CRC 5-year survival with $\geq 70\%$ accuracy. Medical research indicates that these algorithms perform well in similar studies, making it likely that at least one of the models in this project will achieve the aim of $\geq 70\%$ accuracy. This also contributes to the success criterion by providing confidence that the chosen approaches would produce acceptable results for recall, specificity, and AUC.

6. Data Mining Algorithm (DMA)

6.1. Data Mining Algorithm Analysis

The researcher is a second-degree relative of a colorectal cancer patient who died from the disease in 2019. This personal connection motivated the researcher to explore patient survival as the focus of this research. Several datasets were explored and the selected dataset was chosen due to the presence of the binary target variable `Survival_5_yrs`. It has been established within the medical community that classification algorithms suit survival analysis the most. Classification techniques produce interpretable results for clinical decision support (Han et al., 2024; Maawadh et al., 2024). The researcher is interested in exploring how XGBoost, RSF, and Naive Bayes could be used to predict five-year survival outcomes for patients with CRC.

XGBoost is an algorithm that has gained popularity in data science because of its high performance. It is known for making extremely accurate predictions and is frequently utilised in research and competitions. One of its key advantages is that it trains models rapidly and efficiently on massive amounts of data. It can also deal with a variety of input data types, such as numbers and categories, and it handles missing values automatically. XGBoost

offers built-in methods to reduce overfitting that allow the model to remain reliable even when the data is noisy or uneven. These capabilities make it a versatile and powerful choice for data mining tasks that require accuracy and efficiency (NVIDIA, n.d.; Nopour, 2024; Karunakaran et al., 2025). The research could benefit from using XGBoost with its ability to handle massive amounts of data and reduce overfitting in the selected dataset. It could also help in satisfying the data mining objective of achieving $\geq 70\%$ model accuracy based on medical research.

RSF is a machine learning technique that is highly effective in predicting survival times. It works by creating many decision trees and integrating their results, making the predictions more stable and less prone to overfitting. RSF can handle censored data, and it can use records where the full survival time is unknown. It can also identify complex patterns in data, such as how multiple patient factors interact with one another. Another advantage of RSF is that it identifies the key variables for survival. This, in turn, assists researchers and doctors in understanding which factors have a significant impact on outcomes (Wu et al., 2023; Wang & Li, 2017; Steele, 2025). It may be worthwhile to investigate how RSF can enhance the five-year survival prognosis of CRC. Not only does it meet the success requirements of achieving $\geq 70\%$ model accuracy in medical studies, but it can also handle censored data like health records.

Naive Bayes is a simple probability-based technique for making predictions. It is quick to train and simple to use for working with massive datasets. Despite the assumption that all input features are independent, it frequently produces unexpectedly good results in practice. Another advantage is that it performs well with high-dimensional data and it can manage multiple variables at once without being overly complex. Because it delivers data that are simple to read, it is an excellent choice for medical research that requires clear explanations (Pajila et al., 2023). Although Naive Bayes may not always reach the best prediction accuracy, it does provide a solid foundation for comparison. This supports the purpose of training 1-3 models and comparing their performance, as Naïve Bayes allows the researcher to assess whether more advanced algorithms such as XGBoost or RSF significantly improve above the $\geq 70\%$ baseline target.

XGBoost, RSF, and Naïve Bayes provide considerable advantages to the research. XGBoost is useful for working with huge datasets and can help achieve the 70% accuracy objective. RSF is intended for survival data and may be used with censored health records appropriate for medical studies. Naive Bayes' simplicity and ease of explanation make it an excellent starting point for comparing other complex models. Looking at all three combined provides a balanced approach that supports the research goals and prepares for the selection phase in 6.2.

6.2. Data Algorithm Selection

This step determines which algorithms will be used for building the model. This decision is based on how the broad strengths of the algorithms discussed in Section 6.1 correspond to the project's data mining goals and success criteria. These include at least 70% prediction accuracy, preprocessing of the dataset, and evaluating up to three classification models based on accuracy, recall, specificity, and AUC.

C5.0 and Random Tree models were considered in the previous iteration of decision tree algorithms. While these models exhibited the value of interpretable and rule-based predictions, they did not achieve the data mining objectives. This prompted the researcher to broaden the range of algorithms in the current iteration.

XGBoost, RSF, and Naive Bayes have been chosen for further examination due to their potential improvements in five-year survival prediction. XGBoost was chosen for its proven ability to deliver highly accurate predictions with huge and complex datasets such as health records. The dataset currently holds 167497 rows and 24 columns. XGBoost could provide the computational efficiency and predictive strength needed to achieve the $\geq 70\%$ accuracy threshold. RSF was selected since it is primarily designed for survival analysis and performs well with healthcare data. It can process censored records and capture interactions between patient characteristics. RSF offers variable importance metrics that can help researchers discover the most influential predictors of survival. Its ability to balance prediction accuracy and interpretability makes it an algorithm worth exploring for this research. Naive Bayes is used as the baseline classifier. Although simpler than the other two algorithms, it is faster, more efficient, and easier to understand. Adding it enables the researcher to examine if more advanced algorithms, such as XGBoost or RSF, provide substantial improvements in predicting accuracy and other evaluation criteria.

During the model and parameter selection step, the researcher attempted to apply RSF. RSF was initially included in the list of possible models due to its extensive use in medical survival research. However, upon a deeper examination of the dataset, it was discovered that RSF could not be implemented because the data lacked the essential time-to-event and censoring fields. The dataset only includes a binary outcome (Survival_5_yrs). It is consistent with classification methodologies, but not survival modelling. As a result, RSF was excluded from the data algorithm selection.

6.3. Model and Parameter Selection

6.3.1. XGBoost

```
=== Experiment with test_size=0.2 (80/20 split) ===
Accuracy: 0.5992238805970149
ROC AUC: 0.49654107140400094
Confusion Matrix:
[[ 11 13411]
 [ 15 20063]]

Classification Report:
      precision    recall  f1-score   support

   False      0.42      0.00      0.00    13422
    True      0.60      1.00      0.75    20078

 accuracy      0.51      0.50      0.60    33500
  macro avg      0.51      0.50      0.38    33500
 weighted avg      0.53      0.60      0.45    33500

=== Experiment with test_size=0.25 (75/25 split) ===
Accuracy: 0.5991402985074626
ROC AUC: 0.49588558486290496
Confusion Matrix:
[[ 12 16766]
 [ 20 25077]]

Classification Report:
      precision    recall  f1-score   support

   False      0.38      0.00      0.00    16778
    True      0.60      1.00      0.75    25097

 accuracy      0.49      0.50      0.60    41875
  macro avg      0.49      0.50      0.38    41875
 weighted avg      0.51      0.60      0.45    41875
```

*Figure 30: Screenshot of XGBoost baseline parameters and results
With different splits
(refer to [Appendix J](#) for full code snippet)*

The simulations looked at how the model performed when the dataset was divided into different proportions for training and testing. In the first experiment, with an 80/20 split, the model had an accuracy of 0.60 and a ROC AUC of 0.497 resulting in something similar to random guessing. The confusion matrix and classification report revealed that the model expected almost all patients to survive. This resulted in perfect recall for the survivor class (1.00) and zero recall for the non-survivor class (0.00). Precision for the survivor class was 0.60, indicating that 60% of survival forecasts were true. In the second experiment, with a 75/25 split, the accuracy was maintained at 0.60 while the ROC AUC decreased slightly to 0.496. A similar problem came up where the model predicted everything as survival, resulting in perfect recall for the survivor class but none for the non-survivors. These findings demonstrate that adjusting the train-test split did not improve performance. The algorithm remains biased towards predicting survival and rendering it unhelpful for identifying individuals who did not survive.

6.3.2. Naive Bayes

```
=== Experiment with test_size=0.2 (80/20 split) ===
=== Naive Bayes Results ===
Accuracy: 0.5993432835820895
ROC AUC: 0.4973400248492955
Confusion Matrix:
[[ 0 13422]
 [ 0 20078]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	13422
1	0.60	1.00	0.75	20078
accuracy			0.60	33500
macro avg	0.30	0.50	0.37	33500
weighted avg	0.36	0.60	0.45	33500

```


=== Class-conditional feature means (top 10) ===

```

	Class 0	Class 1
Age	69.214256	69.161065
Alcohol_Consumption	0.502738	0.500504
BMI_Obese	0.284831	0.284165
BMI_Overweight	0.428643	0.431645
Cancer_Stage_Metastatic	0.191253	0.190726
Cancer_Stage_Regional	0.429090	0.430898
Diabetes	0.199672	0.201198
Diet_Risk	1.099352	1.100437
Early_Detection	0.598327	0.598501
Family_History	0.303867	0.296916

Figure 31: Screenshot of Naive Bayes baseline parameters and results with 80/20 split

(refer to [Appendix K](#) for full code snippet)

The models examined how the Naive Bayes model performed when the dataset was divided into different proportions for training and testing. In the first experiment, with an 80/20 split, the model had an accuracy of 0.60 and a ROC AUC of 0.497. The confusion matrix and classification report revealed that the model categorised nearly all patients as survivors. This resulted in perfect recall for the survivor class (1.00) and no recall for the non-survivor class (0.00). The precision for the survivor class was 0.60.

```

=== Experiment with test_size=0.25 (75/25 split) ===
=== Naive Bayes Results ===
Accuracy: 0.5993313432835821
ROC AUC: 0.4986104231471745
Confusion Matrix:
[[ 0 16778]
 [ 0 25097]]

Classification Report:
              precision    recall  f1-score   support

     0       0.00        0.00        0.00      16778
     1       0.60        1.00        0.75      25097

 accuracy          0.30        0.50        0.60      41875
  macro avg          0.30        0.50        0.37      41875
 weighted avg          0.36        0.60        0.45      41875

=== Class-conditional feature means (top 10) ===
              Class 0      Class 1
Age          69.202158    69.156382
Alcohol_Consumption    0.503397    0.500159
BMI_Obese          0.284253    0.284619
BMI_Overweight      0.428991    0.431518
Cancer_Stage_Metastatic    0.191667    0.190052
Cancer_Stage_Regional    0.429508    0.431438
Diabetes          0.200072    0.200757
Diet_Risk          1.100274    1.100000
Early_Detection      0.598109    0.598194
Family_History      0.303664    0.296972

```

Figure 32: Screenshot of Naive Bayes baseline parameters and results with 75/25 split
(refer to [Appendix K](#) for full code snippet)

In the second experiment, with a 75/25 split, the accuracy remained at 0.60, while the ROC AUC increased marginally to 0.499. A similar problem happened when the model projected all patients as survivors. This results in perfect recall for the survivor class but none for the non-survivor class. Precision for the survivor class remained at 0.60, with an F1-score of 0.75.

7. Data Mining (DM)

7.1. Data Mining Tests

Logical test strategies were developed to ensure that the models were evaluated fairly and consistently against the study's objectives. Two typical partitioning strategies were used: the 80/20 split and the 75/25 split. The 80/20 split is widely accepted as the classic predictive modelling standard, providing enough data for training while reserving a representative group for testing (Han et al., 2024). The 75/25 split was also adopted, based on evidence from colorectal cancer survival studies that it produces a stronger holdout set for validation while keeping sufficient training data (Buk Cardoso et al., 2023). These designs enabled a comparative study of how the partitioning technique affected model performance and generalisability.

7.2. Running Model

Please refer to the appendices and .py files to see the model run.

7.3. Data Mining Patterns

7.3.1. XGBoost

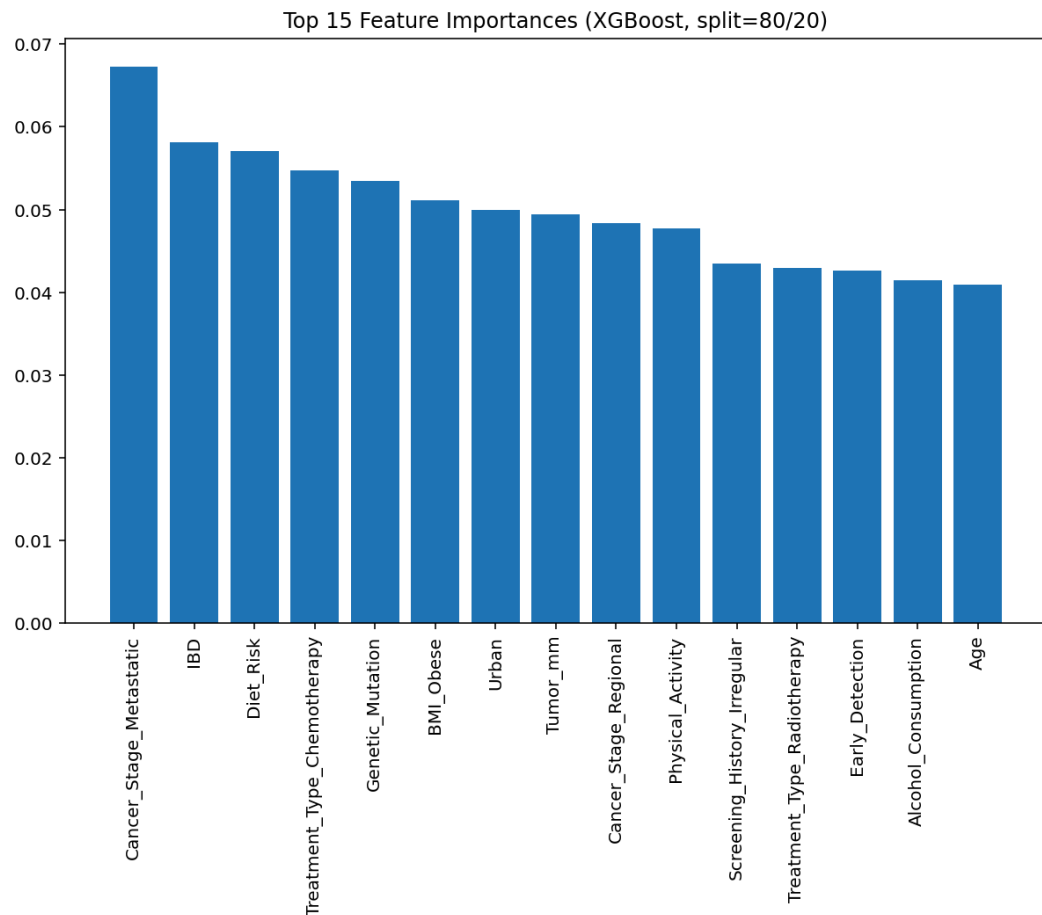


Figure 33: Screenshot of XGBoost patterns for 80/20 split

Figure 32 illustrates the top 15 key attributes of the XGBoost model in an 80/20 split. The most significant factor was Cancer_Stage_Metastatic, followed by IBD, Diet_Risk, Treatment_Type_Chemotherapy, genetic mutation, BMI_Obese, and urban. These factors had the most impact on survival predictions. Lifestyle factors such as alcohol consumption and age came towards the bottom. These attributes had less influence than clinical variables.

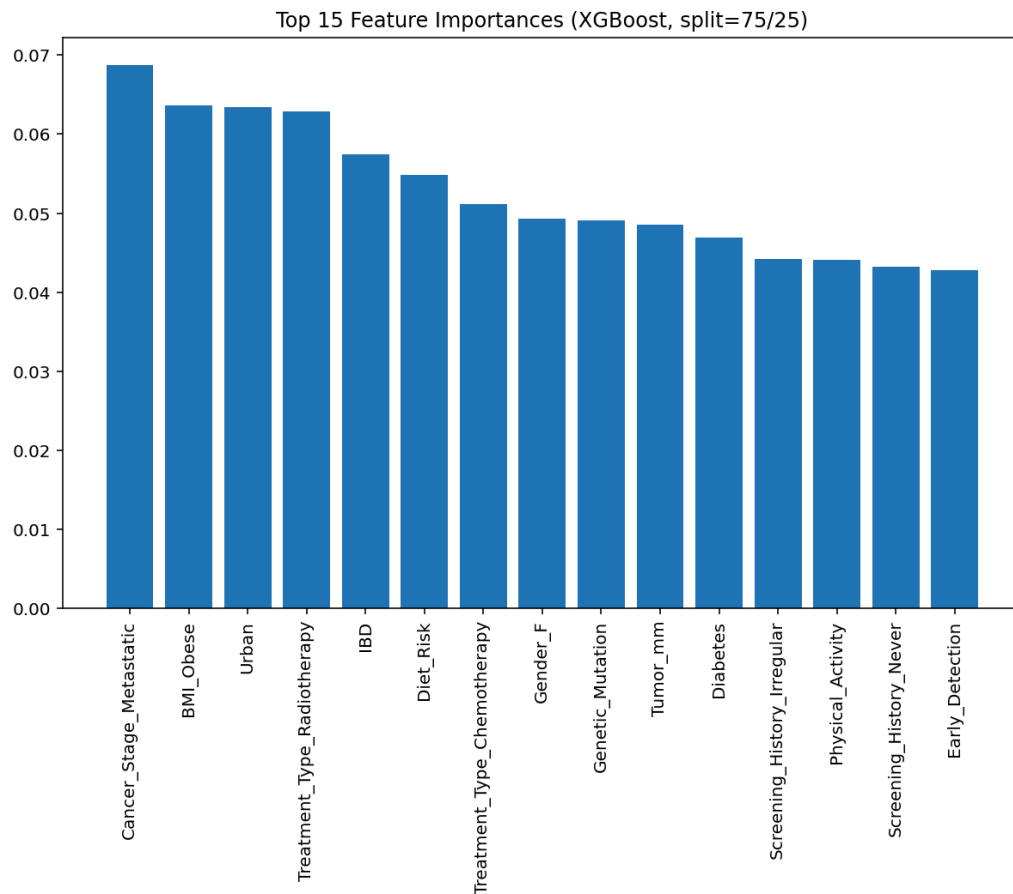


Figure 34: Screenshot of XGBoost patterns for 75/25 split

Figure 33 presents the model's top 15 significant attributes in a 75/25 ratio. Again, Cancer_Stage_Metastatic was the most critical factor, but this time BMI_Obese and urban residence had a greater influence than in the previous experiment. Treatment-related attributes such as radiotherapy and chemotherapy were also highly rated. Additional variables, such as Gender_F and Diabetes, were on this list but were less prevalent in the 80/20 split.

7.3.2. Naive Bayes

```
=== Experiment with test_size=0.2 (80/20 split) ===
=== Naive Bayes Results ===
Accuracy: 0.5993432835820895
ROC AUC: 0.4973400248492955
Confusion Matrix:
[[ 0 13422]
 [ 0 20078]]

Classification Report:
              precision    recall  f1-score   support

     0         0.00         0.00         0.00       13422
     1         0.60         1.00         0.75       20078

 accuracy          0.30          0.50          0.60       33500
 macro avg         0.30          0.50          0.37       33500
 weighted avg      0.36          0.60          0.45       33500

=== Class-conditional feature means (top 10) ===
              Class 0      Class 1
Age          69.214256   69.161065
Alcohol_Consumption  0.502738   0.500504
BMI_Obese     0.284831   0.284165
BMI_Overweight 0.428643   0.431645
Cancer_Stage_Metastatic 0.191253   0.190726
Cancer_Stage_Regional  0.429090   0.430898
Diabetes      0.199672   0.201198
Diet_Risk     1.099352   1.100437
Early_Detection 0.598327   0.598501
Family_History 0.303867   0.296916
```

Figure 35: Screenshot of Naive Bayes with 80/20 split

The Naive Bayes results indicate a persistent prediction bias in the dataset. The 80/20 and 75/25 splits reveal that the model classified nearly all patients as survivors, regardless of their actual outcome. This behaviour is explained by the class-conditional feature means and shows that the average values of survivors and non-survivors are nearly identical. For example, non-survivors had an average age of 69.21 versus 69.16 for survivors, and there were similar overlaps in variables such as BMI_Obese (0.285 vs. 0.284), Diabetes (0.200 vs. 0.201), and Early Detection (0.598 vs. 0.599).

```

=== Experiment with test_size=0.25 (75/25 split) ===
=== Naive Bayes Results ===
Accuracy: 0.5993313432835821
ROC AUC: 0.4986104231471745
Confusion Matrix:
[[ 0 16778]
 [ 0 25097]]

Classification Report:
              precision    recall  f1-score   support

     0           0.00        0.00        0.00       16778
     1           0.60        1.00        0.75       25097

 accuracy          0.30        0.50        0.60       41875
  macro avg          0.30        0.50        0.37       41875
 weighted avg          0.36        0.60        0.45       41875

=== Class-conditional feature means (top 10) ===
              Class 0      Class 1
Age          69.202158    69.156382
Alcohol_Consumption    0.503397    0.500159
BMI_Obese          0.284253    0.284619
BMI_Overweight      0.428991    0.431518
Cancer_Stage_Metastatic 0.191667    0.190052
Cancer_Stage_Regional 0.429508    0.431438
Diabetes           0.200072    0.200757
Diet_Risk           1.100274    1.100000
Early_Detection       0.598109    0.598194
Family_History       0.303664    0.296972

```

Figure 36: Screenshot of Naive Bayes with 75/25 split
(refer to [Appendix K](#) for full code snippet)

These very identical averages indicate that the algorithm was unable to discover significant patterns in the data. Instead, it decided to predict the majority class. This leads to predicting for survivors but not for non-survivors. Naive Bayes' independence assumptions failed to reflect the intricacy of clinical interactions in this dataset. The results show that, while the model is technically sound, its lack of discrimination across classes renders it inappropriate for survival prediction in this context.

8. Interpretation (INT)

8.1. Mined Patterns

The mined patterns provide useful information about both the dataset and the models. The Naive Bayes model has a significant bias towards predicting survival for all patients. The confusion matrix and categorisation report verified this. The recall for the survivor class was 1, whereas the recall for the non-survivor class was 0.00. The accuracy was 0.60, although this figure is deceiving because the model completely ignored one class. The ROC AUC value was close to 0.50. The class-conditional feature provides extra information. The average values for survivors and non-survivors were extremely similar. For non-survivors, the average age was 69.21, while survivors had an average age of 69.16. The same minor changes were observed in BMI_Obese, Diabetes, and Early Detection. The results indicate that Naive Bayes did not detect significant differences between the two groups. This approach assumes that features are independent, yet in health data, many features are interconnected.

The XGBoost model generated more useful patterns. It created feature importance rankings showing which attributes were most important. In both the 80/20 and 75/25 splits, the most important feature was Cancer_Stage_Metastatic. This makes sense because advanced cancer is highly associated with decreased survival. Other key factors included IBD, BMI_Obese, and treatment options such as chemotherapy and radiotherapy. These findings indicate that both health problems and treatment options influence survival. Some lifestyle and demographic characteristics, such as urban and diet risk, had some influence, but not as much as clinical factors.

Although XGBoost identified crucial variables, its overall performance was inadequate. The ROC AUC was again close to 0.50, suggesting a weak separation of survivors and non-survivors. The precision-recall curve indicated that precision remained around 0.60. This indicates that many predictions were inaccurate. The recall rate was high, but it came at the expense of predicting most patients as survivors. This demonstrates that the model has trouble producing balanced predictions. The difficulty stems from the dataset. The survivors and non-survivors have highly similar feature values, therefore the model cannot distinguish between them clearly.

8.2. Data, Results, Models, and Patterns Visualisation

8.2.1. XGBoost

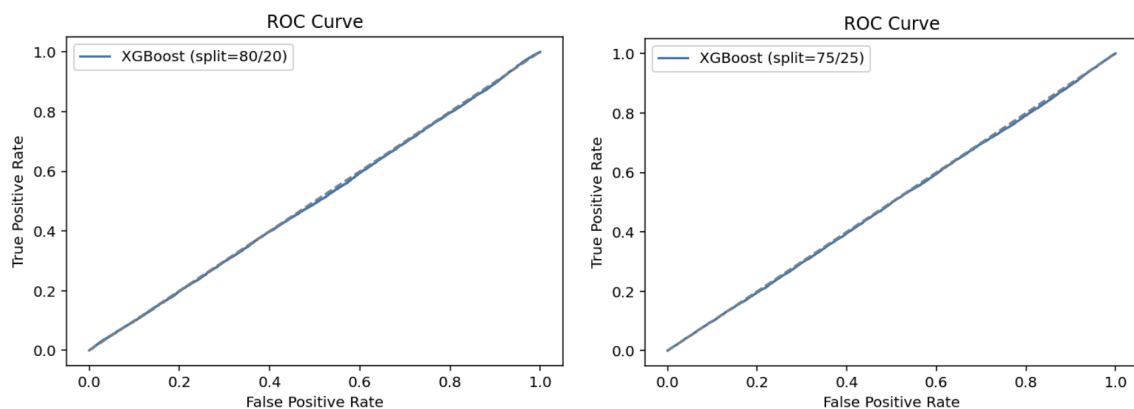


Figure 37: Screenshot of XGBoost ROC curve

Figure 34 illustrate the ROC curves for the XGBoost model with two different train-test splits: 80/20 and 75/25. In both examples, the blue line is almost identical to the diagonal grey line. This suggests that the model is unable to distinguish survivors from non-survivors in a meaningful way. The curves in the two studies are quite similar, indicating that adjusting the split ratio does not enhance the outcome. These visualisations demonstrate that the model is struggling to extract useful patterns from the data.

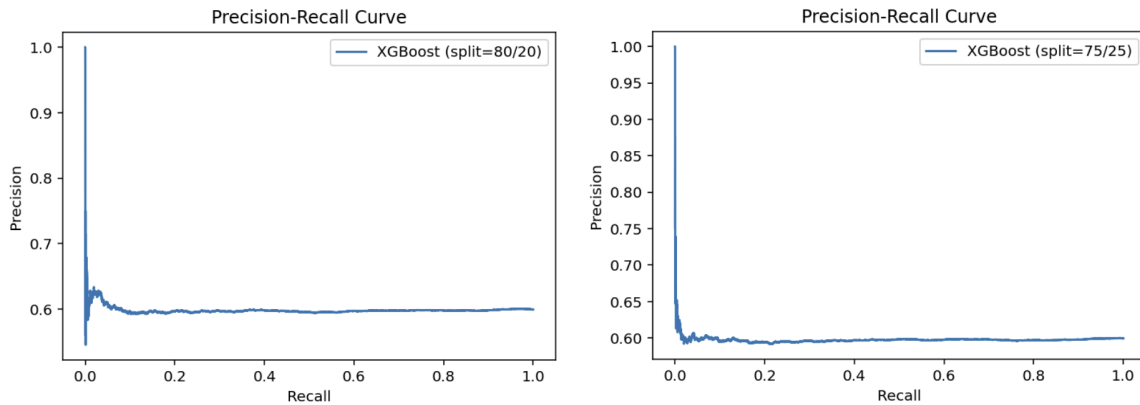


Figure 38: Screenshot of XGBoost Precision-Recall curve

Figure 35 demonstrates how well the XGBoost model combines precision and recall across two train-test splits. In both the 80/20 and 75/25 trials, the curves flatten soon with a precision of approximately 0.60. This means that if recall increases, the model's precision does not improve any more. The shape of the curves also indicates that there is little trade-off between the two measures. In practice, this means that the model is limited. While it detects more true cases, it also generates a large number of inaccurate predictions. The results are essentially the same in both splits. The model's behaviour is stable yet ineffective.

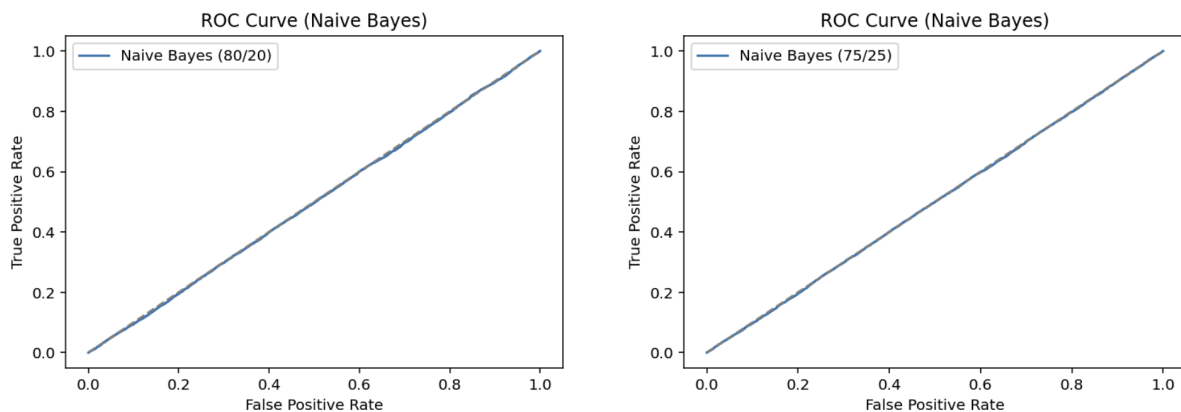


Figure 39: Screenshot of Naive Bayes ROC curve

The figures show the ROC curves for Naive Bayes with an 80/20 and 75/25 splits. In both situations, the curve almost perfectly follows the diagonal line. This demonstrates that the model does not distinguish between the two classes and performs close to random guessing. The fact that the two curves are almost identical suggests that adjusting the train-test split has little to no influence on the model's capacity to learn.

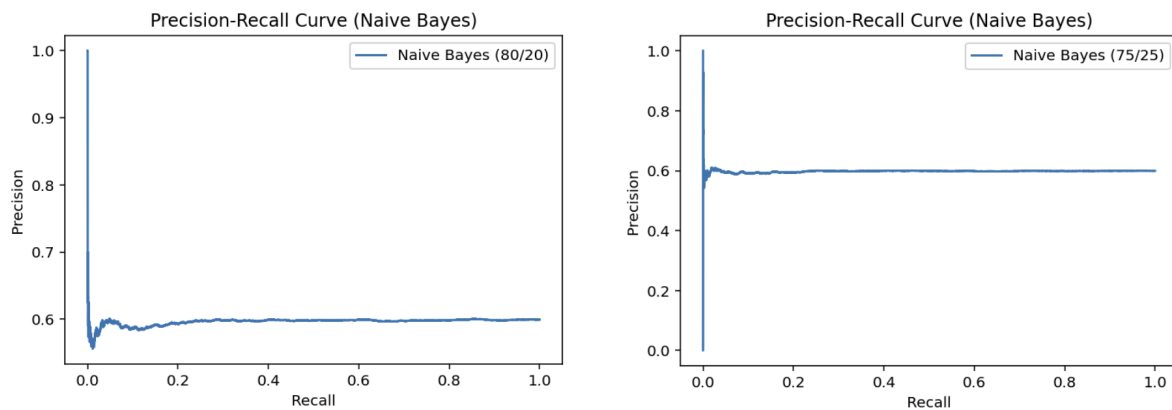


Figure 40: Screenshot of Naive Bayes Precision-Recall curve

Figure 37 shows how Naive Bayes performs in precision and recall for 80/20 and 75/25 splits. In both situations, the curves quickly level out with a precision of approximately 0.60. This demonstrates that the model provides the same proportion of correct and incorrect predictions regardless of the number of cases it attempts to capture. The almost flat appearance of the lines suggests that the model's performance cannot be meaningfully adjusted.

8.3. Results, Models, and Patterns Interpretation

XGBoost and Naïve Bayes failed to correctly predict five-year outcomes for colorectal cancer patients. The models achieved an accuracy of approximately 60%, but this figure is misleading because they classified the majority of patients as survivors. This explains why recall for the survivor class was high while recall for the non-survivor class was zero. In fact, that behaviour reduces the models' usefulness because identifying individuals at increased risk of mortality is frequently more clinically significant than verifying survival. Naive Bayes' assumption of feature independence may be very simplistic for clinical data. The class-conditional means revealed that survivors and non-survivors had nearly identical values for crucial factors such as age, diabetes, BMI, and early diagnosis. Because of the overlap, the algorithm chose to predict the majority class. This explains why the ROC and precision-recall curves for Naive Bayes remained similar to random guessing.

XGBoost provide more detailed information by rating the importance of various traits. Cancer stage, inflammatory bowel disease, obesity, and treatment kinds (such as chemotherapy and radiotherapy) were the most important predictors. These findings are consistent with the medical literature, which states that advanced cancer stage and comorbidities significantly reduce survival chances, whereas access to appropriate treatments improves outcomes. Despite finding relevant characteristics, the model was still unable to effectively distinguish survivors from non-survivors. The ROC curves revealed essentially little improvement, and the precision-recall curves flattened around 0.60 predictions, making it unreliable. These findings indicate that the dataset itself may limit model performance. Survivors and non-survivors appear remarkably similar across many factors, limiting algorithms' capacity to discover significant differences. In other words, the issue is less about the models used and more about the predictive signal in the data. Additional or more detailed variables, such as

genetic markers, cancer biomarkers, or longitudinal treatment data, may be necessary to create a model with an accuracy of 70% or better.

8.4. Assessment and Evaluation of Results, Models, and Patterns

The results demonstrate that neither Naive Bayes nor XGBoost met the success target of 70% accuracy. Both models maintained approximately 60% accuracy, with ROC AUC values close to 0.50. This indicates they were unable to easily distinguish between survivors and non-survivors. Although Naive Bayes is rapid and straightforward to train, it assumes feature independence, which is not the case for medical data. As a result, it classified almost all patients as survivors while failing to identify non-survivors. XGBoost performed better in determining which factors are most important, such as cancer stage, treatment type, and obesity, but it was also unable to make reliable predictions.

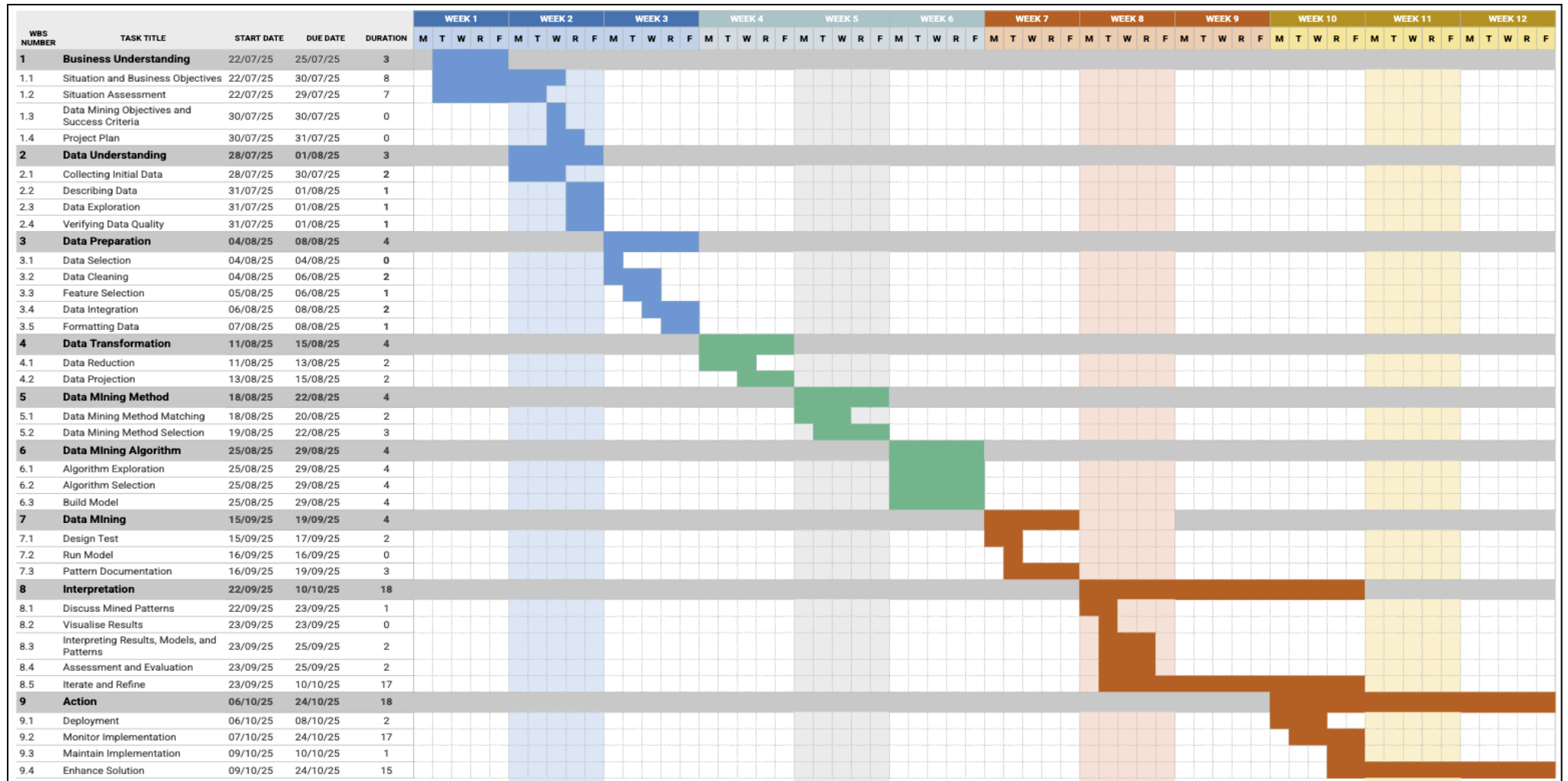
These findings indicate that the problem is less with the algorithms and more with the dataset. Many characteristics, such as age, BMI, and diabetes, had extremely comparable averages in both groups, limiting the models' capacity to differentiate survival outcomes. The imbalance in the target variable also prompted the models to concentrate on survivors. Overall, the models provided useful insights into significant determinants but did not produce solid predicted results. To improve outcomes, future research may require more specific data, such as genetic or treatment histories, or sophisticated approaches for dealing with imbalanced datasets.

8.5. Iterations

During Iteration 3, the researcher went through the CRISP-DM method numerous times. Following feature selection, encoding, and transformations such as Box-Cox, baseline models were constructed using XGBoost and Naive Bayes. The evaluation stage revealed that both models were limited, prompting the researcher to revisit the algorithm selection step and evaluate the classifiers with various train-test splits (80/20 and 75/25). Following modelling and pattern discovery, XGBoost identified essential characteristics such as cancer stage and therapy kind, whereas Naive Bayes produced overlapping feature averages. This iterative procedure helped the researcher better understand the dataset and the challenges of reaching the prediction objectives.

Appendices

Appendix A: Gantt Chart



Appendix B: Describing Colorectal Cancer Dataset

The screenshot displays a Jupyter Notebook environment with a code editor on the left and a console output on the right.

Code Editor (Left):

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Created on Sun Sep 21 23:12:31 2025
5
6 @author: alessonabao
7 """
8
9 # 01-BU
10 """
11 support clinical decision-making in colorectal cancer (CRC) care by
12 developing a predictive model that estimates the likelihood of
13 patient survival beyond five years following diagnosis
14 """
15
16 # 02-DU
17 import pandas as pd
18
19 # load datasets
20 Primary_Dataset = 'colorectal_cancer_dataset.csv'
21 Secondary_Dataset = 'crc_dataset.csv'
22 Tertiary_Dataset = 'crc_dataset1.csv'
23
24 df1 = pd.read_csv(Primary_Dataset)
25 df2 = pd.read_csv(Secondary_Dataset)
26 df3 = pd.read_csv(Tertiary_Dataset)
27
28 # print first and last 3 columns with first and last 5 rows
29 print(df1)
30 print(df2)
31 print(df3)
32
33 # visualise top 5 rows
34 df1.head()
35 df2.head()
36 df3.head()
37
38 # show number of rows and columns of each dataset
39 df1.shape
40 df2.shape
41 df3.shape
42
43 # know key information of dataframes (col, non-null, data type)
44 df1.info()
45 df2.info()
46 df3.info()
47
```

Console Output (Right):

```
In [1]: import pandas as pd
In [2]: Primary_Dataset = 'colorectal_cancer_dataset.csv'
In [3]: df1 = pd.read_csv(Primary_Dataset)
In [4]: df1.shape
Out[4]: (167497, 28)
In [5]: df1.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 167497 entries, 0 to 167496
Data columns (total 28 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   Patient_ID                            167497 non-null  int64
1   Country                               167497 non-null  object
2   Age                                   167497 non-null  int64
3   Gender                               167497 non-null  object
4   Cancer_Stage                          167497 non-null  object
5   Tumor_Size_mm                         167497 non-null  int64
6   Family_History                        167497 non-null  object
7   Smoking_History                       167497 non-null  object
8   Alcohol_Consumption                   167497 non-null  object
9   Obesity_BMI                           167497 non-null  object
10  Diet_Risk                             167497 non-null  object
11  Physical_Activity                     167497 non-null  object
12  Diabetes                              167497 non-null  object
13  Inflammatory_Bowel_Disease             167497 non-null  object
14  Genetic_Mutation                       167497 non-null  object
15  Screening_History                     167497 non-null  object
16  Early_Detection                       167497 non-null  object
17  Treatment_Type                        167497 non-null  object
18  Survival_5_years                       167497 non-null  object
19  Mortality                             167497 non-null  object
20  Healthcare_Costs                       167497 non-null  int64
21  Incidence_Rate_per_100K                167497 non-null  int64
22  Mortality_Rate_per_100K                 167497 non-null  int64
23  Urban_or_Rural                         167497 non-null  object
24  Economic_Classification                 167497 non-null  object
25  Healthcare_Access                       167497 non-null  object
26  Insurance_Status                       167497 non-null  object
27  Survival_Prediction                     167497 non-null  object
dtypes: int64(6), object(22)
memory usage: 35.8+ MB
In [6]:
```

Variable Explorer (Top Right):

Name	Type	Size	Value
df1	DataFrame	(167497, 28)	Column names: Patient_ID, Country, Age, Gender, Cancer_Stage, Tumor_Size ...
Primary_Dataset	str	29	colorectal_cancer_dataset.csv

Bottom Bar: Python Console | History | Inline | Conda: anaconda3 (Python 3.13.6) | LSP: Python | Line 48, Col 1 | UTF-8 | LF | RW | Mem: 72%

Appendix C: Checking for null and missing data

The screenshot displays a Jupyter Notebook environment with a code editor on the left and a console on the right.

Code Editor (2DataUnderstanding.py):

```
106 # -----2.3 - Data validation-----
107 # check missing values
108 # Count missing values per column
109 # ++++++SCREENSHOT+++++
110 missing_per_col = df1.isnull().sum()
111 print(df1.isnull().sum())
112
113 # Percentage of missing values per column
114 # ++++++SCREENSHOT+++++
115 percent_missing_per_col = (df1.isnull().mean() * 100).round(2)
116 print((df1.isnull().mean() * 100).round(2))
117
118 # Duplication check
119 # ++++++SCREENSHOT+++++
120 print("Duplicate rows:", df1.duplicated().sum())
121
122 # check data imbalance
123 # Distribution of target
124 sns.countplot(x="Survival_5_years", data=df1)
125 plt.title("Class Distribution: Survival (5 Years)")
126 plt.show()
127
128 # check for outliers
129 def iqr_outliers(series):
130     Q1 = series.quantile(0.25)
131     Q3 = series.quantile(0.75)
132     IQR = Q3 - Q1
133     lower_bound = Q1 - 1.5 * IQR
134     upper_bound = Q3 + 1.5 * IQR
135     outliers = series[(series < lower_bound) | (series > upper_bound)]
136     return outliers, lower_bound, upper_bound
137
138 # outliers in Age
139 outlier_age, lower, upper = iqr_outliers(df1["Age"])
140 print(f"Outliers in Age: {len(outlier_age)} values")
141 print(f"Lower bound: {lower}, Upper bound: {upper}")
142
143 # outliers in Tumor_Size_mm
144 outlier_tumor_size, lower, upper = iqr_outliers(df1["Tumor_Size_mm"])
145 print(f"Outliers in Tumor_Size_mm: {len(outlier_tumor_size)} values")
146 print(f"Lower bound: {lower}, Upper bound: {upper}")
147
148 # Visualise
149 sns.boxplot(x=df1["Age"])
150 plt.title("Outliers in Age (IQR Method)")
151 plt.show()
```

Console (Console 1/A):

```
In [5]: missing_per_col = df1.isnull().sum()
...: print(df1.isnull().sum())
Patient_ID      0
Country         0
Age            0
Gender         0
Cancer_Stage   0
Tumor_Size_mm  0
Family_History 0
Smoking_History 0
Alcohol_Consumption 0
Obesity_BMI     0
Diet_Risk       0
Physical_Activity 0
Diabetes        0
Inflammatory_Bowel_Disease 0
Genetic_Mutation 0
Screening_History 0
Early_Detection 0
Treatment_Type  0
Survival_5_years 0
Mortality       0
Healthcare_Costs 0
Incidence_Rate_per_100K 0
Mortality_Rate_per_100K 0
Urban_or_Rural  0
Economic_Classification 0
Healthcare_Access 0
Insurance_Status 0
Survival_Prediction 0
dtype: int64

In [6]: percent_missing_per_col = (df1.isnull().mean() * 100).round(2)
...: print((df1.isnull().mean() * 100).round(2))
Patient_ID      0.0
Country         0.0
Age            0.0
Gender         0.0
Cancer_Stage   0.0
Tumor_Size_mm  0.0
dtype: object
```

History (history.py):

```
import matplotlib.pyplot as plt
missing_per_col = df1.isnull().sum()
print(df1.isnull().sum())
percent_missing_per_col = (df1.isnull().mean() * 100).round(2)
print((df1.isnull().mean() * 100).round(2))
print("Duplicate rows:", df1.duplicated().sum())
```

Status Bar: Inline | Conda: anaconda3 (Python 3.13.5) | LSP: Python | main [1] | Line 125, Col 45 | UTF-8 | LF | RW | Mem 75%

Appendix D: Checking for duplicates and outliers

The screenshot displays a Jupyter Notebook environment with a dark theme. The main editor on the left contains a Python script named `2DataUnderstanding.py`. The script includes comments and code for data validation, such as checking for missing values and duplicates. It also defines a function `iqr_outliers` to identify outliers using the Interquartile Range (IQR) method. The script checks for outliers in the 'Age' and 'Tumor_Size_mm' columns and visualizes the results using boxplots.

```
106 # -----2.3 - Data validation-----
107 # check missing values
108 # Count missing values per column
109 # ++++++SCREENSHOT+++++
110 missing_per_col = df1.isnull().sum()
111 print(df1.isnull().sum())
112
113 # Percentage of missing values per column
114 # ++++++SCREENSHOT+++++
115 percent_missing_per_col = (df1.isnull().mean() * 100).round(2)
116 print((df1.isnull().mean() * 100).round(2))
117
118 # Duplication check
119 # ++++++SCREENSHOT+++++
120 print("Duplicate rows:", df1.duplicated().sum())
121
122 # check data imbalance
123 # Distribution of target
124 sns.countplot(x="Survival_5_years", data=df1)
125 plt.title("Class Distribution: Survival (5 Years)")
126 plt.show()
127
128 # check for outliers
129 def iqr_outliers(series):
130     Q1 = series.quantile(0.25)
131     Q3 = series.quantile(0.75)
132     IQR = Q3 - Q1
133     lower_bound = Q1 - 1.5 * IQR
134     upper_bound = Q3 + 1.5 * IQR
135     outliers = series[(series < lower_bound) | (series > upper_bound)]
136     return outliers, lower_bound, upper_bound
137
138 # outliers in Age
139 outlier_age, lower, upper = iqr_outliers(df1["Age"])
140 print(f"Outliers in Age: {len(outlier_age)} values")
141 print(f"Lower bound: {lower}, Upper bound: {upper}")
142
143 outlier_tumor_size, lower, upper = iqr_outliers(df1["Tumor_Size_mm"])
144 print(f"Outliers in Tumor_Size_mm: {len(outlier_tumor_size)} values")
145 print(f"Lower bound: {lower}, Upper bound: {upper}")
146
147 # Visualise
148 sns.boxplot(x=df1["Age"])
149 plt.title("Outliers in Age (IQR Method)")
150 plt.show()
151
```

The right sidebar shows the 'Console 1/A' output, which displays the results of the code execution. It shows the number of missing values per column, the percentage of missing values, the number of duplicate rows, and the results of the outlier detection for 'Age' and 'Tumor_Size_mm'. The console also shows the execution of the boxplot for 'Age'.

```

Mortality_Rate_per_100k      0.0
Urban_or_Rural               0.0
Economic_Classification      0.0
Healthcare_Access            0.0
Insurance_Status              0.0
Survival_Prediction           0.0
dtype: float64

In [7]: print("Duplicate rows:", df1.duplicated().sum())
Duplicate rows: 0

In [8]: def iqr_outliers(series):
...:     Q1 = series.quantile(0.25)
...:     Q3 = series.quantile(0.75)
...:     IQR = Q3 - Q1
...:     lower_bound = Q1 - 1.5 * IQR
...:     upper_bound = Q3 + 1.5 * IQR
...:     outliers = series[(series < lower_bound) | (series > upper_bound)]
...:     return outliers, lower_bound, upper_bound
...:
...: # outliers in Age
...: outlier_age, lower, upper = iqr_outliers(df1["Age"])
...: print(f"Outliers in Age: {len(outlier_age)} values")
...: print(f"Lower bound: {lower}, Upper bound: {upper}")
...:
...: outlier_tumor_size, lower, upper = iqr_outliers(df1["Tumor_Size_mm"])
...: print(f"Outliers in Tumor_Size_mm: {len(outlier_tumor_size)} values")
...: print(f"Lower bound: {lower}, Upper bound: {upper}")
Outliers in Age: 0 values
Lower bound: 29.0, Upper bound: 109.0
Outliers in Tumor_Size_mm: 0 values
Lower bound: -34.0, Upper bound: 118.0

In [9]: sns.boxplot(x=df1["Age"])
...: plt.title("Outliers in Age (IQR Method)")
...: plt.show()

In [10]:

```

The bottom status bar indicates the environment is Conda: anaconda3 (Python 3.13.5) with LSP: Python, main [1], Line 150, Col 11, UTF-8, LF, RW, and Mem 75%.

Appendix E: Data cleaning

The screenshot displays a Jupyter Notebook environment with a dark theme. The main window shows a Python script named `3DataPreparation.py` with the following code:

```
72 df_selected.head()
73 # -----3.2 - data cleaning-----
74 # checks
75 # ---nulls---
76 # +-----+SCREENSHOT+-----+
77 print("Missing values per column before imputation:")
78 print(df_selected[cols_with_nulls].isnull().sum())
79 print("Selected shape: ", df_selected.shape)
80
81 continuous_cols = ['Age', 'Tumor_Size_mm']
82 categorical_cols = ['Cancer_Stage', 'Obesity_BMI', 'Physical_Activity']
83
84 for col in continuous_cols:
85     median_val = df_selected[col].median()
86     df_selected.fillna({col: median_val}, inplace=True)
87     print(f"Imputed {col} with median value {median_val}")
88
89 for col in categorical_cols:
90     mode_val = df_selected[col].mode()[0]
91     df_selected.fillna({col: mode_val}, inplace=True)
92     print(f"Imputed {col} with mode value '{mode_val}'")
93
94 # +-----+SCREENSHOT+-----+
95 print("\nMissing values per column AFTER imputation:")
96 print(df_selected[cols_with_nulls].isnull().sum())
97 print("Selected shape: ", df_selected.shape)
98
99 # ---white spaces---
100 # +-----+SCREENSHOT+-----+
101 obj_cols = df_selected.select_dtypes(include=['object']).columns.tolist()
102
103 for col in obj_cols:
104     # count how many entries have leading or trailing spaces
105     space_issues = df_selected[col].astype(str).apply(
106         lambda x: (x != x.strip()))
107     .sum()
108
109     if space_issues > 0:
110         print(f"{col}: {space_issues} values with leading/trailing spaces")
111     else:
112         print(f"{col}: no space issues detected")
113
114 # ---duplicates---
115 # +-----+SCREENSHOT+-----+
116 # other columns will have repeated categorical values
117 for col in df_selected.columns:
118     counts = df_selected[col].value_counts()
119     duplicates = counts[counts > 1]
120     print(f"\nColumn: {col}")
121     if not duplicates.empty:
122         print(f"  Number of duplicated values: {len(duplicates)}")
123         print(f"  Top duplicated values:")
124         print(duplicates.head(5))
125     else:
126         print(f"  No duplicates found (all values unique).")
127 # -----3.3 - data construct-----
```

The right-hand pane shows the `Console 1/A` output, which displays the results of the data cleaning process:

```
Missing values per column before imputation:
Age      8375
Tumor_Size_mm  8375
Cancer_Stage  8375
Obesity_BMI  8375
Physical_Activity  8375
dtype: int64
Selected shape: (167497, 22)
Imputed Age with median value 69.0
Imputed Tumor_Size_mm with median value 42.0
Imputed Cancer_Stage with mode value 'Regional'
Imputed Obesity_BMI with mode value 'Overweight'
Imputed Physical_Activity with mode value 'Moderate'

Missing values per column AFTER imputation:
Age      0
Tumor_Size_mm  0
Cancer_Stage  0
Obesity_BMI  0
Physical_Activity  0
dtype: int64
Selected shape: (167497, 22)
Gender: no space issues detected
Cancer_Stage: no space issues detected
Family_History: no space issues detected
Smoking_History: no space issues detected
Alcohol_Consumption: no space issues detected
Obesity_BMI: no space issues detected
Diet_Risk: no space issues detected
Physical_Activity: no space issues detected
Diabetes: no space issues detected
Inflammatory_Bowel_Disease: no space issues detected
Genetic_Mutation: no space issues detected
Screening_History: no space issues detected
Early_Detection: no space issues detected
Treatment_Type: no space issues detected
Survival_5_years: no space issues detected
Urban_or_Rural: no space issues detected
Healthcare_Access: no space issues detected
Insurance_Status: no space issues detected
```

Below the console output, there are tabs for `IPython Console`, `Help`, `Variable Explorer`, `Debugger`, `Plots`, and `Files`. The `history.py` tab is active, showing the execution history of the code:

```
if not duplicates.empty:
    print(f"  Number of duplicated values: {len(duplicates)}")
    print(f"  Top duplicated values:")
    print(duplicates.head(5))
else:
    print(f"  No duplicates found (all values unique).")
```

The bottom status bar indicates the environment is `Conda: anaconda3 (Python 3.13.5)`, the language server is `LSP: Python`, the file is `main`, the cursor is at `Line 127, Col 33`, the encoding is `UTF-8`, the line ending is `LF`, the right margin is `RW`, and memory usage is `Mem 70%`.

Appendix F: Data construct

The screenshot displays a JupyterLab environment with a Python script in the left pane and a variable explorer in the right pane.

Python Script (3DataPreparation.py):

```
123 print(" Top duplicated values:")
124 print(duplicates.head(5))
125 else:
126     print(" No duplicates found (all values unique).")
127 # ----3.3 - data construct----
128 # split data into different patient data categories
129 KEY, TARGET = "Patient_ID", "Survival_5_years"
130
131 patient_demographics = df_selected[[KEY, "Age", "Gender"]].copy()
132
133 patient_clinical = df_selected[[KEY,
134     "Cancer_Stage", "Tumor_Size_mm", "Family_History",
135     "Diabetes", "Inflammatory_Bowel_Disease", "Genetic_Mutation"
136     ]].copy()
137
138 patient_lifestyle = df_selected[[KEY,
139     "Smoking_History", "Alcohol_Consumption", "Obesity_BMI",
140     "Diet_Risk", "Physical_Activity"
141     ]].copy()
142
143 patient_treatment = df_selected[[KEY, "Treatment_Type", "Screening_History", \
144     "Early_Detection"]].copy()
145
146 # split Urban_or_Rural into urban and rural and Insurance_Status into insured and uninsured
147 # One-hot encode Urban_or_Rural into 'urban' and 'rural' columns
148 # ++++++SCREENSHOT+++++
149 urban_dummies = pd.get_dummies(
150     df_selected["Urban_or_Rural"].str.lower(),
151     prefix="",
152     prefix_sep=""
153 )[[ "urban", "rural" ]]
154
155 insured_dummies = pd.get_dummies(
156     df_selected["Insurance_Status"].str.lower(),
157     prefix="",
158     prefix_sep=""
159 )[[ "insured", "uninsured" ]] # keep only these two columns
160
161 patient_access = pd.concat(
162     [
163         df_selected[[KEY, "Healthcare_Access"]],
164         insured_dummies, urban_dummies
165     ],
166     axis=1
167 ).copy()
168
169 patient_costs = df_selected[[KEY, "Healthcare_Costs"]].copy()
170
171 patient_outcomes = df_selected[[KEY, TARGET]].copy()
172 # ----3.4 - data integration----
173 # merge patient_demographics, patient_clinical, patient_lifestyle,
174 # patient_treatment, patient_access, patient_costs
175 # patient_outcomes into one dataset again
176 # ++++++SCREENSHOT+++++
```

Variable Explorer:

Name	Type	Size	Value
patient_access	DataFrame	[167497, 6]	Column names: Patient_ID, Heal
patient_clinical	DataFrame	[167497, 7]	Column names: Patient_ID, Canc
patient_costs	DataFrame	[167497, 2]	Column names: Patient_ID, Heal
patient_demographics	DataFrame	[167497, 3]	Column names: Patient_ID, Age,
patient_lifestyle	DataFrame	[167497, 6]	Column names: Patient_ID, Smok
patient_outcomes	DataFrame	[167497, 2]	Column names: Patient_ID, Surv
patient_treatment	DataFrame	[167497, 4]	Column names: Patient_ID, Trea
Primary_Dataset	str	29	colorectal_cancer_dataset.csv
space_issues	int64	1	np.int64(0)
TARGET	str	16	Survival_5_years
urban_dummies	DataFrame	[167497, 2]	Column names: urban, rural

history.py:

```
prefix="",
prefix_sep=""
) [[ "urban", "rural" ]]

insured_dummies = pd.get_dummies(
    df_selected["Insurance_Status"].str.lower(),
    prefix="",
    prefix_sep=""
) [[ "insured", "uninsured" ]] # keep only these two columns

patient_access = pd.concat(
    [
        df_selected[[KEY, "Healthcare_Access"]],
        insured_dummies, urban_dummies
    ],
    axis=1
).copy()
```

Status Bar: Conda: anaconda3 (Python 3.13.5) ✓ LSP: Python V main Line 171, Col 53 UTF-8 LF RW Mem 68%

Appendix G: Data integration

The screenshot displays a JupyterLab environment with a dark theme. The left pane shows a Python script named `3DataPreparation.py` with line numbers 172 to 226. The script performs data integration by merging several datasets (`patient_demographics`, `patient_clinical`, `patient_lifestyle`, `patient_treatment`, `patient_access`, `patient_costs`, and `patient_outcomes`) into a single `df_integrated` DataFrame. It then reorders columns, selects specific columns for a new dataset, and checks for duplicates and null values. The right pane shows the console output for the script, which includes the following text:

```
Integrated dataset shape: (167497, 24)
Selected dataset shape: (167497, 22)
Original dataset shape: (167497, 28)
Rows in final dataset: 167497
Unique Patient IDs: 167497
No duplicate Patient IDs
Missing Patient IDs: 0
Total null values in dataset: 0
```

The bottom status bar indicates the environment is using Conda: anaconda3 (Python 3.13.5) with LSP: Python, and the current file is `main` at Line 224, Col 56. The memory usage is 70%.

Appendix H: Data reformatting

```
225 # -----3.5 - data reformatting-----
226
227 df_final = df_integrated.copy()
228
229 # remove Patient_ID since it's not needed anymore
230 df_final = df_final.drop(columns=["Patient_ID"])
231
232 # rename long columns
233 rename_cols = {
234     "Tumor_Size_mm": "Tumor_mm",
235     "Obesity_BMI": "BMI",
236     "Inflammatory_Bowel_Disease": "IBD",
237     "Survival_5_years": "Survival_5_yrs",
238     "Healthcare_Costs": "HC_Costs",
239     "Urban": "Urban",
240     "rural": "Rural",
241     "insured": "Insured",
242     "uninsured": "Uninsured"
243 }
244
245 df_final.rename(columns={k: v for k, v in rename_cols.items() if k in df_final.columns}, inplace=True)
246
247 # change yes/no values to true/false for consistency
248 yes_no_cols = ['Family_History', 'Smoking_History', 'Alcohol_Consumption', \
249               'Diabetes', 'IBD', 'Genetic_Mutation', 'Early_Detection', \
250               'Survival_5_yrs']
251
252 def yesno_to_bool(series: pd.Series) -> pd.Series:
253     # Map yes/no (and 1/0, true/false strings) to booleans
254     s = series.astype(str).str.strip().str.lower()
255     mapped = s.map({
256         "yes": True, "y": True, "true": True, "1": True,
257         "no": False, "n": False, "false": False, "0": False
258     })
259     # Use pandas Nullable Boolean dtype to allow missing values (pd.NA)
260     return mapped.astype("boolean")
261
262 for col in yes_no_cols:
263     if col in df_final.columns:
264         df_final[col] = yesno_to_bool(df_final[col])
265
266 # check changes
267 print("Boolean dtypes check:")
268 print(df_final[yes_no_cols].dtypes) # should show 'boolean'
269
270 print("\nValue checks (first 5 rows):")
271 print(df_final[yes_no_cols].head())
272
273 print("\nCounts of True/False/NA per column:")
274 for c in yes_no_cols:
275     if c in df_final.columns:
276         print(c, df_final[c].value_counts(dropna=False).to_dict())
277
278 # apply one-hot encoding
279 one_hot_encode_cols = ["Gender", "Cancer_Stage", "BMI", "Screening_History", "Treatment_Type"]
```

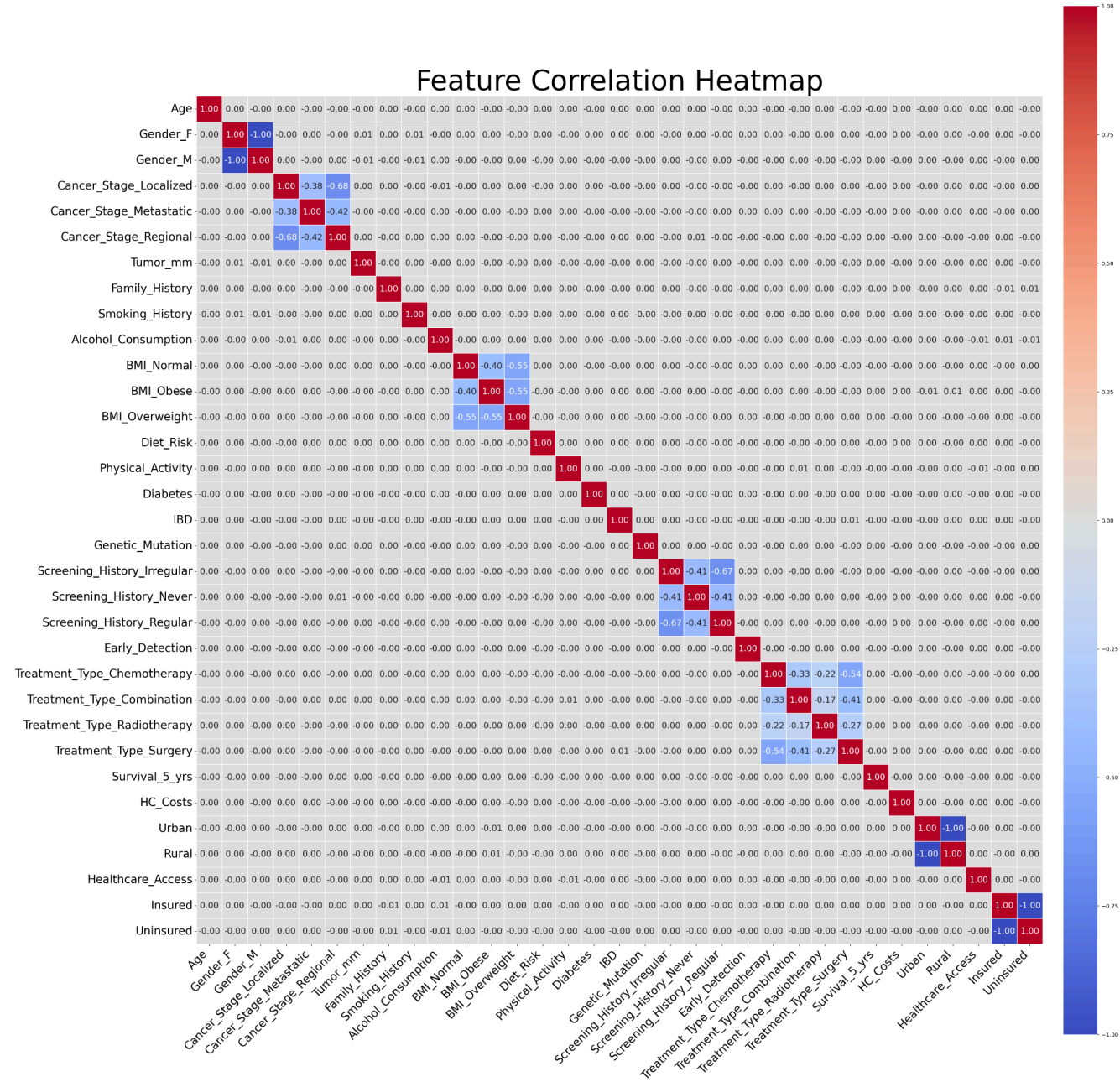
```
278 # apply one-hot encoding
279 one_hot_encode_cols = ["Gender", "Cancer_Stage", "BMI", "Screening_History", "Treatment_Type"]
280
281 df_final = pd.get_dummies(df_final, columns=one_hot_encode_cols, drop_first=False)
282
283 dummy_cols = [c for c in df_final.columns if any(col + "_" in c for col in one_hot_encode_cols)]
284 df_final[dummy_cols] = df_final[dummy_cols].astype("Int64")
285
286 # apply label encoding
287 ordinal_map = {
288     "low": 0,
289     "moderate": 1,
290     "high": 2
291 }
292
293 ordinal_cols = ["Diet_Risk", "Physical_Activity", "Healthcare_Access"]
294
295 for col in ordinal_cols:
296     if col in df_final.columns:
297         df_final[col] = (
298             df_final[col]
299             .astype(str)
300             .str.strip()
301             .str.lower()
302             .map(ordinal_map)
303             .astype("Int64") # nullable integer, preserves missing if any
304         )
305
306 # reorder columns similar to original dataset
307 column_order = [
308     "Age",
309     # Gender dummies
310     *[c for c in df_final.columns if c.startswith("Gender_")],
311     # Cancer_Stage dummies
312     *[c for c in df_final.columns if c.startswith("Cancer_Stage_")],
313     "Tumor_mm",
314     "Family_History",
315     "Smoking_History",
316     "Alcohol_Consumption",
317     # BMI dummies
318     *[c for c in df_final.columns if c.startswith("BMI_")],
319     "Diet_Risk",
320     "Physical_Activity",
321     "Diabetes",
322     "IBD",
323     "Genetic_Mutation",
324     # Screening_History dummies
325     *[c for c in df_final.columns if c.startswith("Screening_History_")],
326     "Early_Detection",
327     # Treatment_Type dummies
328     *[c for c in df_final.columns if c.startswith("Treatment_Type_")],
329     "Survival_5_yrs",
330     "HC_Costs",
331     "Urban", "Rural", "Healthcare_Access", "Insured", "Uninsured"
332 ]
```

Appendix H continuation

```
332     "Urban", "Rural", "Healthcare_Access", "Insured", "Uninsured"
333 ]
334
335 column_order = [c for c in column_order if c in df_final.columns]
336
337 df_final = df_final[column_order].copy()
338
339 # make sure datatypes are set to the correct type
340 continuous_cols = [c for c in ["Age", "Tumor_mm", "HC_Costs"] if c in df_final.columns]
341 for col in continuous_cols:
342     df_final[col] = pd.to_numeric(df_final[col], errors="coerce").astype("Int64")
343
344 ordinal_cols = [c for c in ["Diet_Risk", "Physical_Activity", "Healthcare_Access"] if c in df_final.columns]
345 for col in ordinal_cols:
346     df_final[col] = pd.to_numeric(df_final[col], errors="coerce").astype("Int8")
347
348 bool_cols = [c for c in [
349     "Family_History", "Smoking_History", "Alcohol_Consumption",
350     "Diabetes", "IBD", "Genetic_Mutation", "Early_Detection",
351     "Survival_5_yrs"
352 ] if c in df_final.columns]
353
354 for col in bool_cols:
355     # If any residual strings slipped through, normalise; else astype will be a no-op
356     s = df_final[col]
357     if s.dtype == object:
358         s = (s.astype(str).str.strip().str.lower()
359             .map({"yes": True, "y": True, "true": True, "1": True,
360                 "no": False, "n": False, "false": False, "0": False}))
361     df_final[col] = s.astype("boolean") # pandas nullable Boolean (allows <NA>)
362
363 dummy_prefixes = ["Gender_", "Cancer_Stage_", "BMI_", "Screening_History_", "Treatment_Type_"]
364 dummy_cols = [c for c in df_final.columns if any(c.startswith(p) for p in dummy_prefixes)]
365 for col in dummy_cols:
366     df_final[col] = pd.to_numeric(df_final[col], errors="coerce").astype("Int8")
367
368 for col in [c for c in ["Urban", "Rural", "Insured", "Uninsured"] if c in df_final.columns]:
369     df_final[col] = pd.to_numeric(df_final[col], errors="coerce").astype("Int8")
370
371 # check if there are object type cols remaining
372 residual_object = [c for c in df_final.columns if df_final[c].dtype == object]
373 if residual_object:
374     print("Note: residual object-typed columns:", residual_object)
375
376 # check all types and nulls
377 print("\n=== 3.5 DTYPE SUMMARY ===")
378 print(df_final.dtypes.sort_index())
379
380 print("\nNulls per column (top 20):")
381 print(df_final.isnull().sum().sort_values(ascending=False).head(20))
382
383 # # Assert dummy columns are truly 0/1 (or NA)
384 if dummy_cols:
385     dmax = df_final[dummy_cols].max(numeric_only=True).max()
386     dmin = df_final[dummy_cols].min(numeric_only=True).min()
387     assert (dmin in (0, np.nan)) and (dmax in (1, np.nan))
```

```
350     "Diabetes", "IBD", "Genetic_Mutation", "Early_Detection",
351     "Survival_5_yrs"
352 ] if c in df_final.columns]
353
354 for col in bool_cols:
355     # If any residual strings slipped through, normalise; else astype will be a no-op
356     s = df_final[col]
357     if s.dtype == object:
358         s = (s.astype(str).str.strip().str.lower()
359             .map({"yes": True, "y": True, "true": True, "1": True,
360                 "no": False, "n": False, "false": False, "0": False}))
361     df_final[col] = s.astype("boolean") # pandas nullable Boolean (allows <NA>)
362
363 dummy_prefixes = ["Gender_", "Cancer_Stage_", "BMI_", "Screening_History_", "Treatment_Type_"]
364 dummy_cols = [c for c in df_final.columns if any(c.startswith(p) for p in dummy_prefixes)]
365 for col in dummy_cols:
366     df_final[col] = pd.to_numeric(df_final[col], errors="coerce").astype("Int8")
367
368 for col in [c for c in ["Urban", "Rural", "Insured", "Uninsured"] if c in df_final.columns]:
369     df_final[col] = pd.to_numeric(df_final[col], errors="coerce").astype("Int8")
370
371 # check if there are object type cols remaining
372 residual_object = [c for c in df_final.columns if df_final[c].dtype == object]
373 if residual_object:
374     print("Note: residual object-typed columns:", residual_object)
375
376 # check all types and nulls
377 print("\n=== 3.5 DTYPE SUMMARY ===")
378 print(df_final.dtypes.sort_index())
379
380 print("\nNulls per column (top 20):")
381 print(df_final.isnull().sum().sort_values(ascending=False).head(20))
382
383 # # Assert dummy columns are truly 0/1 (or NA)
384 if dummy_cols:
385     dmax = df_final[dummy_cols].max(numeric_only=True).max()
386     dmin = df_final[dummy_cols].min(numeric_only=True).min()
387     assert (dmin in (0, np.nan)) and (dmax in (1, np.nan)), \
388         "Dummy columns contain values other than 0/1."
389
390 # check
391 print("Final shape: ", df_final.shape)
392 print("Final null: ", df_final.isnull().sum())
393 print("Final types: ", df_final.dtypes)
394
395 # export df_final as .csv
396 output_csv = "colorectal_cancer_final.csv"
397
398 # Export with UTF-8 encoding and without the index column
399 df_final.to_csv(output_csv, index=False, encoding="utf-8")
400
401 print(f"Final dataset exported successfully to {output_csv}")
402 print("Shape:", df_final.shape)
```

Appendix I: Correlation heatmap before reduction



Appendix J: XGBoost Model

The screenshot displays a Jupyter Notebook environment with the following components:

- Code Editor:** Contains the script `678DM_XGBoost.py`. The code imports `pandas`, `numpy`, and `matplotlib.pyplot`. It loads the `colorectal_cancer_reduced.csv` dataset and splits it into training and testing sets using `train_test_split`. An XGBoost classifier is trained with the following parameters: `objective="binary:logistic"`, `n_estimators=100`, `max_depth=3`, `learning_rate=0.1`, `random_state=42`, and `eval_metric="auc"`. The model is then fitted to the training data.
- Console:** Shows the output of the code execution. It includes the accuracy, ROC AUC, and confusion matrix for two different test splits: 80/20 and 75/25. The 80/20 split results are as follows:

Experiment with test_size=0.2 (80/20 split)				
	precision	recall	f1-score	support
False	0.42	0.00	0.00	13422
True	0.60	1.00	0.75	20078
accuracy			0.60	33500
macro avg	0.51	0.50	0.38	33500
weighted avg	0.53	0.60	0.45	33500

The 75/25 split results are as follows:

Experiment with test_size=0.25 (75/25 split)				
	precision	recall	f1-score	support
False	0.38	0.00	0.00	16778
True	0.60	1.00	0.75	25097
accuracy			0.60	41875
macro avg	0.49	0.50	0.38	41875
weighted avg	0.51	0.60	0.45	41875
- Plot History:** Shows the code for plotting a Precision-Recall Curve, including `plt.xlabel("Recall")`, `plt.ylabel("Precision")`, `plt.title("Precision-Recall Curve")`, `plt.legend()`, and `plt.show()`.

The bottom status bar indicates the environment is `Conda: anaconda3 (Python 3.13.5)`, the kernel is `LSP: Python`, and the current file is `main [2]` at `Line 98, Col 15`. The encoding is `UTF-8`, the line ending is `LF`, the read-only status is `RW`, and the memory usage is `Mem 78%`.

Appendix J continuation

The screenshot displays a Jupyter Notebook environment with a dark theme. The main editor shows a Python script for training and evaluating an XGBoost model. The script includes code for training the model, predicting on a test set, evaluating performance using accuracy, ROC AUC, and confusion matrix, and generating feature importance plots (bar chart, ROC curve, and Precision-Recall curve).

```
52     eval_metric="auc"           # track AUC
53 )
54
55 # Train the model
56 model.fit(X_train, y_train)
57
58 # Predict on test set
59 y_pred = model.predict(X_test)
60 y_proba = model.predict_proba(X_test)[:, 1]
61
62 # Evaluate performance
63 print("Accuracy:", accuracy_score(y_test, y_pred))
64 print("ROC AUC:", roc_auc_score(y_test, y_proba))
65 print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
66 print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
67
68 # =====
69 importances = model.feature_importances_
70 sorted_idx = np.argsort(importances)[::-1]
71
72 plt.figure(figsize=(10,6))
73 plt.bar(range(len(sorted_idx[:15])), importances[sorted_idx[:15]])
74 plt.xticks(range(len(sorted_idx[:15])), X.columns[sorted_idx[:15]], rotation=90)
75 plt.title(f"Top 15 Feature Importances (XGBoost, split={int((1-split)*100)}/{int(split*100)})")
76 plt.show()
77
78 # =====
79 # ROC Curve
80 fpr, tpr, _ = roc_curve(y_test, y_proba)
81 plt.figure()
82 plt.plot(fpr, tpr, label=f"XGBoost (split={int((1-split)*100)}/{int(split*100)})")
83 plt.plot([0,1],[0,1], "--", color="grey")
84 plt.xlabel("False Positive Rate")
85 plt.ylabel("True Positive Rate")
86 plt.title("ROC Curve")
87 plt.legend()
88 plt.show()
89
90 # Precision-Recall Curve
91 prec, rec, _ = precision_recall_curve(y_test, y_proba)
92 plt.figure()
93 plt.plot(rec, prec, label=f"XGBoost (split={int((1-split)*100)}/{int(split*100)})")
94 plt.xlabel("Recall")
95 plt.ylabel("Precision")
96 plt.title("Precision-Recall Curve")
97 plt.legend()
98 plt.show()
99
```

The right sidebar contains a console window showing the output of the script for two different test set splits. The first split is 0.2 (80/20 split) and the second is 0.25 (75/25 split). The console output includes accuracy, ROC AUC, confusion matrix, and classification report for both splits.

Experiment with test_size=0.2 (80/20 split) ==

Accuracy: 0.5992238805970149
ROC AUC: 0.4965410714040094
Confusion Matrix:
[[11 13411]
 [15 20063]]

	precision	recall	f1-score	support
False	0.42	0.00	0.00	13422
True	0.60	1.00	0.75	20078
accuracy			0.60	33500
macro avg	0.51	0.50	0.38	33500
weighted avg	0.53	0.60	0.45	33500

Experiment with test_size=0.25 (75/25 split) ==

Accuracy: 0.5991402985074626
ROC AUC: 0.49588558486290496
Confusion Matrix:
[[12 16766]
 [20 25077]]

	precision	recall	f1-score	support
False	0.38	0.00	0.00	16778
True	0.60	1.00	0.75	25097
accuracy			0.60	41875
macro avg	0.49	0.50	0.38	41875

The bottom status bar shows the current environment: Conda: anaconda3 (Python 3.13.5), LSP: Python, main [2], Line 30, Col 34, UTF-8, LF, RW, Mem 79%.

Appendix K: Naive Bayes Model

The screenshot displays a Jupyter Notebook environment with the following components:

- Code Editor:** Contains a Python script for a Naive Bayes model. The script defines features, splits the data into training and testing sets, trains a `GaussianNB` model, and evaluates its performance using accuracy, ROC AUC, and a confusion matrix. It also prints class-conditional feature means for the top 10 features.
- Console:** Displays the output of the code execution. It shows the results of two experiments with different test splits (0.2 and 0.25), including accuracy, ROC AUC, and a confusion matrix.
- Variable Explorer:** Shows the variable `feature_means` as a `DataFrame` object.

Code Snippet (678DM_naive_bayes.py):

```
19 # =====6.3=====
20 import pandas as pd
21 import matplotlib.pyplot as plt
22
23 from sklearn.model_selection import train_test_split
24 from sklearn.naive_bayes import GaussianNB
25 from sklearn.metrics import (
26     accuracy_score, roc_auc_score, confusion_matrix, classification_report,
27     roc_curve, precision_recall_curve
28 )
29
30 # Define features and target
31 X = df_reduced.drop(columns=["Survival_5_yrs"]) # predictors
32 y = df_reduced["Survival_5_yrs"].astype(int) # target (0 = no, 1 = yes)
33
34 # =====experiments with different splits =====
35 for split in [0.2, 0.25]: # 80/20 and 75/25
36     print(f"\n=== Experiment with test_size={split} ({int((1-split)*100)}/{int(split*100)}) ===")
37
38     # Train-test split
39     X_train, X_test, y_train, y_test = train_test_split(
40         X, y, test_size=split, random_state=42, stratify=y
41     )
42
43     model = GaussianNB()
44     model.fit(X_train, y_train)
45
46     # Predictions
47     y_pred = model.predict(X_test)
48     y_proba = model.predict_proba(X_test)[:, 1]
49
50     # Evaluation
51     print("=== Naive Bayes Results ===")
52     print("Accuracy:", accuracy_score(y_test, y_pred))
53     print("ROC AUC:", roc_auc_score(y_test, y_proba))
54     print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
55     print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
56
57     # =====7=====
58     # Naive Bayes does not provide feature importance directly
59     feature_means = pd.DataFrame(model.theta_, columns=X.columns, index=["Class 0", "Class 1"])
60     print("\n=== Class-conditional feature means (top 10) ===")
61     print(feature_means.head(10))
62
63     # =====8=====
64     # ROC Curve
65     fpr, tpr, _ = roc_curve(y_test, y_proba)
66     plt.figure()
```

Console Output (Console 1/A):

```
=== Experiment with test_size=0.2 (80/20 split) ===
=== Naive Bayes Results ===
Accuracy: 0.5993432835820895
ROC AUC: 0.4973400248492955
Confusion Matrix:
[[ 0 13422]
 [ 0 20078]]

Classification Report:
              precision    recall  f1-score   support

      0       0.00        0.00        0.00    13422
      1       0.60        1.00        0.75    20078

 accuracy          0.60        0.60        0.60    33500
 macro avg         0.30        0.50        0.37    33500
 weighted avg      0.36        0.60        0.45    33500

=== Class-conditional feature means (top 10) ===
              Class 0      Class 1
Age          69.214256    69.161065
Alcohol_Consumption  0.502738    0.500504
BMI_Obese       0.284831    0.284165
BMI_Overweight  0.428643    0.431645
Cancer_Stage_Metastatic  0.191253    0.190726
Cancer_Stage_Regional  0.429090    0.430098
Diabetes        0.199672    0.201198
Diet_Risk       1.099352    1.100437
Early_Detection  0.598327    0.598501
Family_History  0.303867    0.296916

=== Experiment with test_size=0.25 (75/25 split) ===
=== Naive Bayes Results ===
Accuracy: 0.5993313432835821
ROC AUC: 0.4986104231471745
Confusion Matrix:
[[ 0 16778]
 [ 0 25097]]
```

Variable Explorer:

- `feature_means`: `DataFrame` object

Code Snippet (history.py):

```
plt.plot(rec, prec, label=f"Naive Bayes ({int((1-split)*100)}/{int(split*100)})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve (Naive Bayes)")
plt.legend()
plt.show()
```

Status Bar: Inline | Conda: anaconda3 (Python 3.13.5) | LSP: Python | main [2] | Line 83, Col 15 | UTF-8 | LF | RW | Mem 78%

Appendix K continuation

The screenshot displays a JupyterLab environment with a Python script in the editor and its execution results in the console.

Script File: 678DM_naive_bayes.py

```
36 print(f"\n== Experiment with test_size={split} ({int((1-split)*100)}/{int(spl
37
38 # Train-test split
39 X_train, X_test, y_train, y_test = train_test_split(
40     X, y, test_size=split, random_state=42, stratify=y
41 )
42
43 model = GaussianNB()
44 model.fit(X_train, y_train)
45
46 # Predictions
47 y_pred = model.predict(X_test)
48 y_proba = model.predict_proba(X_test)[: , 1]
49
50 # Evaluation
51 print("=== Naive Bayes Results ===")
52 print("Accuracy:", accuracy_score(y_test, y_pred))
53 print("ROC AUC:", roc_auc_score(y_test, y_proba))
54 print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
55 print("\nClassification Report:\n", classification_report(y_test, y_pred, zero
56
57 # =====7=====
58 # Naive Bayes does not provide feature importance directly
59 feature_means = pd.DataFrame(model.theta_, columns=X.columns, index=["Class 0"
60 print("\n== Class-conditional feature means (top 10) ==")
61 print(feature_means.head(10))
62
63 # =====8=====
64 # ROC Curve
65 fpr, tpr, _ = roc_curve(y_test, y_proba)
66 plt.figure()
67 plt.plot(fpr, tpr, label=f"Naive Bayes ({int((1-split)*100)}/{int(split*100)})
68 plt.plot([0,1],[0,1],"-",color="grey")
69 plt.xlabel("False Positive Rate")
70 plt.ylabel("True Positive Rate")
71 plt.title("ROC Curve (Naive Bayes)")
72 plt.legend()
73 plt.show()
74
75 # Precision-Recall Curve
76 prec, rec, _ = precision_recall_curve(y_test, y_proba)
77 plt.figure()
78 plt.plot(rec, prec, label=f"Naive Bayes ({int((1-split)*100)}/{int(split*100)})
79 plt.xlabel("Recall")
80 plt.ylabel("Precision")
81 plt.title("Precision-Recall Curve (Naive Bayes)")
82 plt.legend()
83 plt.show()
```

Console 1/A Output:

```
=== Experiment with test_size=0.25 (75/25 split) ===
=== Naive Bayes Results ===
Accuracy: 0.5993313432835821
ROC AUC: 0.4986104231471745
Confusion Matrix:
[[ 0 16778]
 [ 0 25097]]

Classification Report:
              precision    recall  f1-score   support

     0       0.00         0.00         0.00         16778
     1       0.60         1.00         0.75         25097

 accuracy          0.30         0.50         0.60         41875
 macro avg         0.30         0.50         0.37         41875
 weighted avg      0.36         0.60         0.45         41875

=== Class-conditional feature means (top 10) ===
              Class 0      Class 1
Age          69.202158    69.156382
Alcohol_Consumption  0.503397    0.500159
BMI_Obese        0.284253    0.284619
BMI_Overweight   0.428991    0.431518
Cancer_Stage_Metastatic  0.191667    0.190052
Cancer_Stage_Regional  0.429508    0.431438
Diabetes         0.200072    0.200757
Diet_Risk        1.100274    1.100000
Early_Detection   0.598109    0.598194
Family_History    0.303664    0.296972
```

history.py

```
plt.plot(rec, prec, label=f"Naive Bayes ({int((1-split)*100)}/{int(split*100)})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve (Naive Bayes)")
plt.legend()
plt.show()
```

Status Bar: Inline Conda: anaconda3 (Python 3.13.5) ✓ LSP: Python Y main [2] Line 32, Col 50 UTF-8 LF RW Mem 77%

References

- Abdul Rahman, H., Ottom, M. A., & Dinov, I. D. (2023). Machine learning-based colorectal cancer prediction using global dietary data. *BMC Cancer*, 23(1). <https://doi.org/10.1186/s12885-023-10587-x>
- Alatise, O. I., Ayandipo, O. O., Adeyeye, A., Seier, K., Komolafe, A. O., Bojuwoye, M. O., Afuwape, O. O., Zauber, A., Omisore, A., Olatoke, S., Akere, A., Famurewa, O., Gonen, M., Irabor, D. O., & Kingham, T. P. (2018). A symptom-based model to predict colorectal cancer in low-resource countries: Results from a prospective study of patients at high risk for colorectal cancer. *Cancer*, 124(13), 2766–2773. <https://doi.org/10.1002/cncr.31399>
- Bass, A. J., Meyerhardt, J. A., Chan, J. A., Giovannucci, E. L., & Fuchs, C. S. (2008). Family history and survival after colorectal cancer diagnosis. *Cancer*, 112(6), 1222–1229. <https://doi.org/10.1002/cncr.23294>
- Chao, A., Thun, M. J., Jacobs, E. J., Henley, S. J., Rodriguez, C., & Calle, E. E. (2000). *Cigarette Smoking and Colorectal Cancer Mortality in the Cancer Prevention Study II*. <https://doi.org/https://doi.org/10.1093/jnci/92.23.1888>
- Duan, B., Zhao, Y., Bai, J., Wang, J., Duan, X., Luo, X., Zhang, R., Pu, Y., Kou, M., Lei, J., & Yang, S. (2022). *Colorectal Cancer: An Overview*. <https://doi.org/https://doi.org/10.36255/exon-publications-gastrointestinal-cancers-colorectal-cancer>
- Green, B. B., & Meenan, R. T. (2020). *Colorectal Cancer Screening- The Costs and Benefits of Getting to 80% in Every Community*. 126(18), 4110–4113. <https://doi.org/https://doi.org/10.1002/cncr.32990>
- Han, Z., Yang, H., Qiao, Q., Wu, T., He, X., & Wang, N. (2024). The survival prediction of advanced colorectal cancer received neoadjuvant therapy—a study of SEER database. *World Journal of Surgical Oncology*, 22(1). <https://doi.org/10.1186/s12957-024-03458-7>
- Heisser, T., Sergeev, D., Hoffmeister, M., & Brenner, H. (2024). Contributions of early detection and cancer prevention to colorectal cancer mortality reduction by screening colonoscopy: a validated modeling study. *Gastrointestinal Endoscopy*, 100(4), 710-717.e9. <https://doi.org/10.1016/j.gie.2024.03.010>
- Klimeck, L., Heisser, T., Hoffmeister, M., & Brenner, H. (2023). Colorectal cancer: A health and economic problem. In *Best Practice and Research: Clinical Gastroenterology* (Vol. 66). Bailliere Tindall Ltd. <https://doi.org/10.1016/j.bpg.2023.101839>
- Lewandowska, A., Rudzki, G., Lewandowski, T., Strykowska-Góra, A., & Rudzki, S. (2022). Risk Factors for the Diagnosis of Colorectal Cancer. *Cancer Control*, 29. <https://doi.org/10.1177/10732748211056692>

Long, D. (2022). *Presented by the EU Centre and Social and Global Studies Centre Jean Monnet Sustainable Development Goals Network Policy Brief Series SDG 3: Ensure healthy lives and promote well-being for all at all ages Health and Development*. www.rmit.edu.au/sdgnetwork

Lu, B., Luo, J., Yan, Y., Zhang, Y., Luo, C., Li, N., Zhou, Y., Wu, D., Dai, M., & Chen, H. (2024). Evaluation of long-term benefits and cost-effectiveness of nation-wide colorectal cancer screening strategies in China in 2020–2060: a modelling analysis. *The Lancet Regional Health - Western Pacific*, 51. <https://doi.org/10.1016/j.lanwpc.2024.101172>

Matsuda, T., Fujimoto, A., & Igarashi, Y. (2025). Colorectal Cancer: Epidemiology, Risk Factors, and Public Health Strategies. In *Digestion* (Vol. 106, Issue 2, pp. 91–99). S. Karger AG. <https://doi.org/10.1159/000543921>

Maawadh, R. M., Xu, C., Ahmed, R., & Mushtaq, N. (2024). Predicting Survival Among Colorectal Cancer Patients: Development and Validation of Polygenic Survival Score. *Clinical and Experimental Gastroenterology*, 17, 317–329. <https://doi.org/10.2147/CEG.S464324>

Mengual-Ballester, M., Pellicer-Franco, E., Valero-Navarro, G., Soria-Aledo, V., García-Marín, J. A., & Aguayo-Albasini, J. L. (2016). Increased survival and decreased recurrence in colorectal cancer patients diagnosed in a screening programme. *Cancer Epidemiology*, 43, 70–75. <https://doi.org/10.1016/j.canep.2016.06.003>

Mohammadi, G., Looha, M. A., Pourhoseingholi, M. A., Tavirani, M. R., Sohrabi, S., Khaneh, A. Z. S., Piri, H., Alaei, T., Parvani, N., Vakilzadeh, I., javadi, S., Cheshmeh, Z. M. H., Razzaghi, Z., Robati, R. M., Azodi, M. Z., Shahraki, S. Z., Talebi, R., Yazdani, J. C., Motlagh, M. E., ... Hadavi, M. (2024). Classification and Diagnostic Prediction of Colorectal Cancer Mortality Based on Machine Learning Algorithms: A Multicenter National Study. *Asian Pacific Journal of Cancer Prevention*, 25(1), 333–342. <https://doi.org/10.31557/APJCP.2024.25.1.333>

Mülder, D. T., van den Puttelaar, R., Meester, R. G. S., O'Mahony, J. F., & Lansdorp-Vogelaar, I. (2023). Development and validation of colorectal cancer risk prediction tools: A comparison of models. *International Journal of Medical Informatics*, 178. <https://doi.org/10.1016/j.ijmedinf.2023.105194>

Nopour, R. (2024). Development of Prediction Model for 5-year Survival of Colorectal Cancer. *Cancer Informatics*, 23. <https://doi.org/10.1177/11769351241275889>

Ning, B., Chi, J., Meng, Q., & Jia, B. (2024). Accurate prediction of colorectal cancer diagnosis using machine learning based on immunohistochemistry pathological images. *Scientific Reports*, 14(1). <https://doi.org/10.1038/s41598-024-76083-9>

Sharp, L., McDevitt, J., Brown, C., Carsin, A., & Comber, H. (2017). *Association between smoking at diagnosis and cause-specific survival in patients with rectal cancer_ Results*

from a population-based analysis of 10,794 cases. 123(13), 2543–2550.
<https://doi.org/https://doi.org/10.1002/cncr.30583>

Sharples, K., Firth, M., Hinder, V., Hill, A., Jee, M., Sarfati, D., Brown, C., Atmore, C., Lawrenson, R., Reid, P., Derrett, S., Macapagal, J., Keating, J., Secker, A., de Groot, C., Jackson, C., & Findlay, M. (2018). The New Zealand PIPER Project: colorectal cancer survival according to rurality, ethnicity and socioeconomic deprivation-results from a retrospective cohort study. *NZMJ*, 8, 1476. www.nzma.org.nz/journal

Walter, V., Jansen, L., Hoffmeister, M., & Brenner, H. (2014). Smoking and survival of colorectal cancer patients: Systematic review and meta-analysis. In *Annals of Oncology* (Vol. 25, Issue 8, pp. 1517–1525). Oxford University Press.
<https://doi.org/10.1093/annonc/mdu040>

Working with text data – pandas 2.3.2 documentation. (n.d.)
https://pandas.pydata.org/pandas-docs/stable/user_guide/text.html#text-types

World Health Organization. (2025, February 3). *Cancer*.
<https://www.who.int/news-room/fact-sheets/detail/cancer>

Yang, X., Qiu, H., Wang, L., & Wang, X. (2023). Predicting Colorectal Cancer Survival Using Time-to-Event Machine Learning: Retrospective Cohort Study. *Journal of Medical Internet Research*, 25. <https://doi.org/10.2196/44417>

Ye, P., Xi, Y., Huang, Z., & Xu, P. (2020). Linking obesity with colorectal cancer: Epidemiology and mechanistic insights. In *Cancers* (Vol. 12, Issue 6). MDPI AG.
<https://doi.org/10.3390/cancers12061408>

Zheng, Y., Hua, X., Win, A. K., MacInnis, R. J., Gallinger, S., le Marchand, L., Lindor, N. M., Baron, J. A., Hopper, J. L., Dowty, J. G., Antoniou, A. C., Zheng, J., Jenkins, M. A., & Newcomb, P. A. (2020). A new comprehensive colorectal cancer risk prediction model incorporating family history, personal characteristics, and environmental factors. *Cancer Epidemiology Biomarkers and Prevention*, 29(3), 549–557.
<https://doi.org/10.1158/1055-9965.EPI-19-0929>

Zhu, B., Wu, X., Wu, B., Pei, D., Zhang, L., & Wei, L. (2017). The relationship between diabetes and colorectal cancer prognosis: A meta-analysis based on the cohort studies. *PLoS ONE*, 12(4). <https://doi.org/10.1371/journal.pone.0176068>

Disclaimer

I acknowledge that the submitted work is my own original work in accordance with the University of Auckland guidelines and policies on academic integrity and copyright. (See:

<https://www.auckland.ac.nz/en/students/forms-policies-and-guidelines/student-policies-and-guidelines/academic-integrity-copyright.html>).

I also acknowledge that I have appropriate permission to use the data that I have utilised in this project. (For example, if the data belongs to an organisation and the data has not been published in the public domain then the data must be approved by the rights holder.) This includes permission to upload the data file to Canvas. The University of Auckland bears no responsibility for the student's misuse of data.